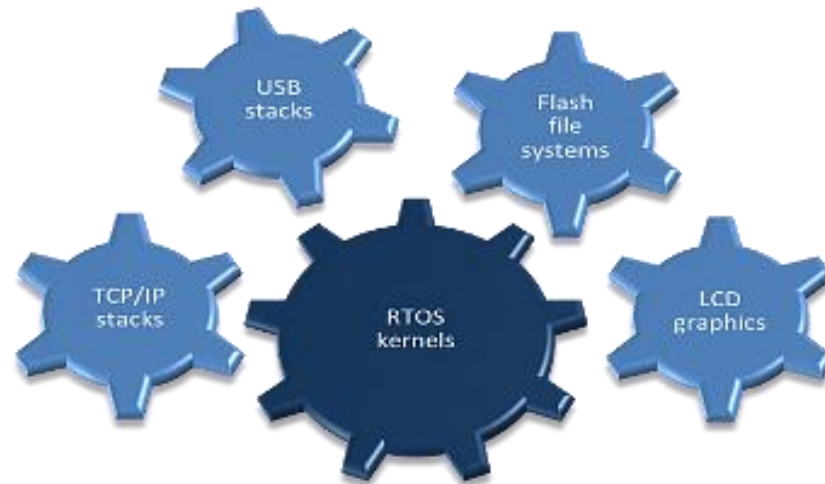
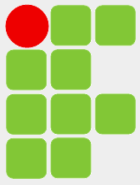


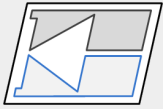
Real Time Operating System



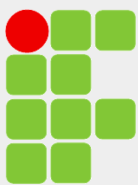
Prof. Charles Borges de Lima.



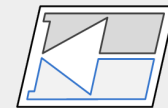
SUMÁRIO



- Introdução
- Sist. Operacional
- Fluxo de Programa
- RTOS
- Tarefas
- TCB
- Preempção
- *Clock Tick*
- Escalonador
- Comunicação e Sincronização
- Exemplo Geral
- Seleção do RTOS
- Conclusão



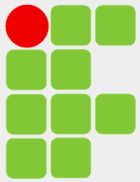
INTRODUÇÃO



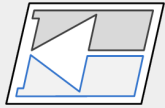
Quando um sistema embarcado exige uma programação que realize inúmeras tarefas em um determinado espaço de tempo, um laço infinito torna difícil a programação. Nesses casos é interessante o emprego de um RTOS.

O uso de um RTOS representa uma forma mais elegante de programação, gerando códigos mais estruturados. Também, melhora o gerenciamento e reuso do código.

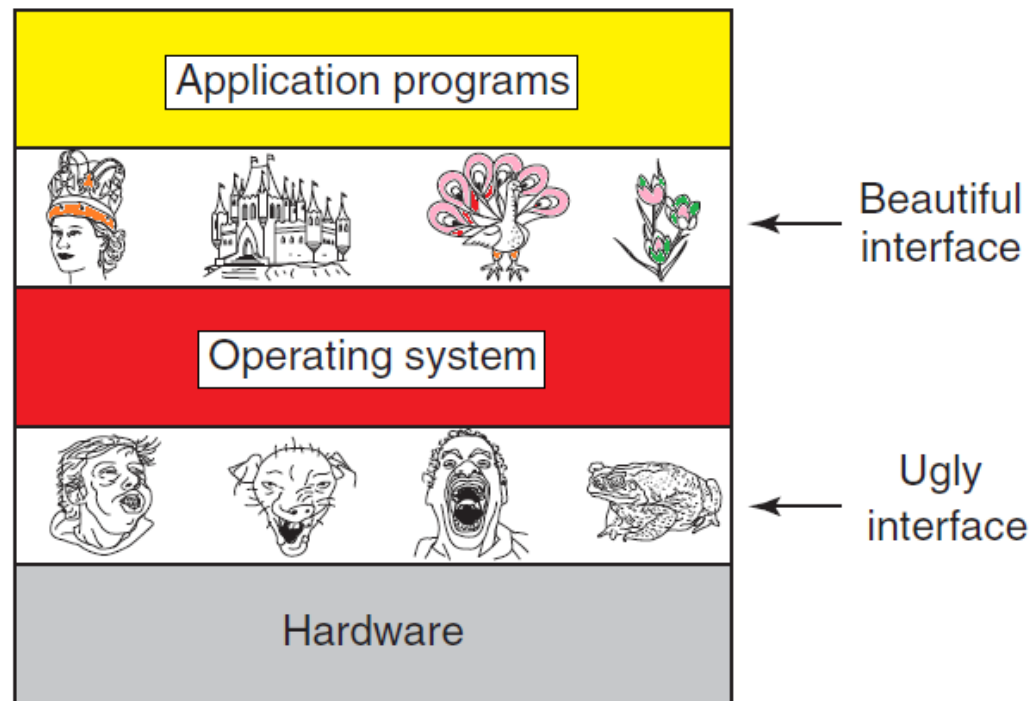
Por outro lado, um RTOS tem necessidade do uso de mais memória e aumenta a latência nas resposta das interrupções do sistema. Como atualmente os μ controladores modernos possuem um bom desempenho e memória, a aplicação é que determinará o emprego ou não de um RTOS.



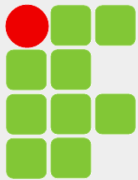
O que é um Sistema Operacional?



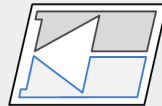
É um software empregado para a simplificação do uso do hardware. Gerencia atividades complexas do hardware sendo um nível de abstração entre o hardware e o usuário.



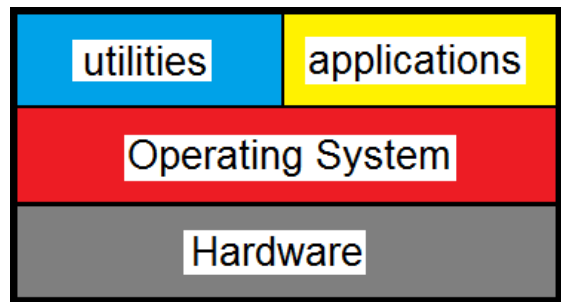
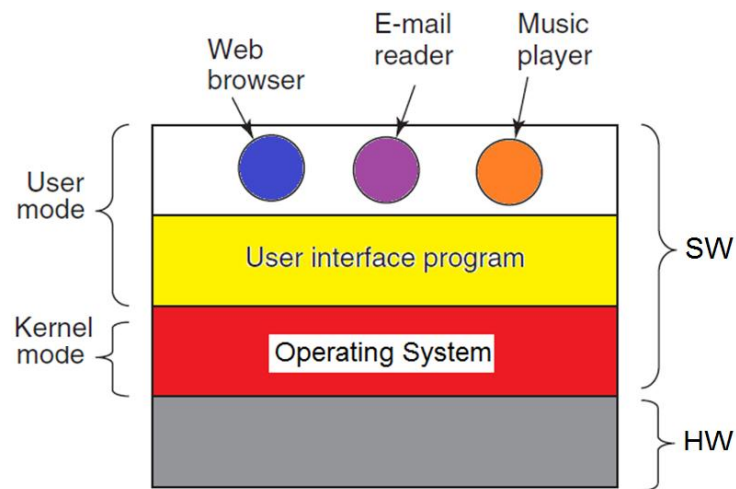
Turn ugly HW into beautiful abstraction.



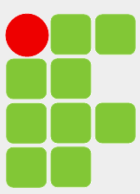
O que é um Sistema Operacional?



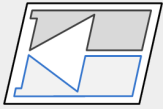
- Coleção de funções (*system calls*).
- Provê um conjunto de serviços básicos para interagir com o HW. É o gerenciador das atividades do sistema.
- O núcleo de um SO é o *Kernel*.
 - Tipicamente uma biblioteca ou conjunto de bibliotecas (funções)
- Tende a ser pequeno e altamente otimizado.



- Pode ou não ter serviços de alto nível no topo:
 - Interface gráfica, I/O, interface do usuário, *network*, etc.

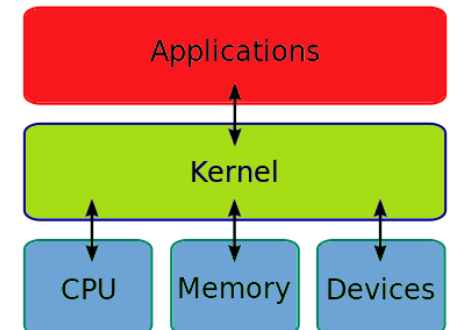
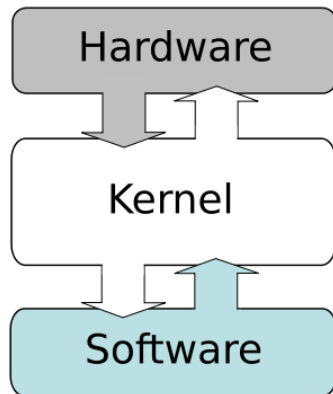
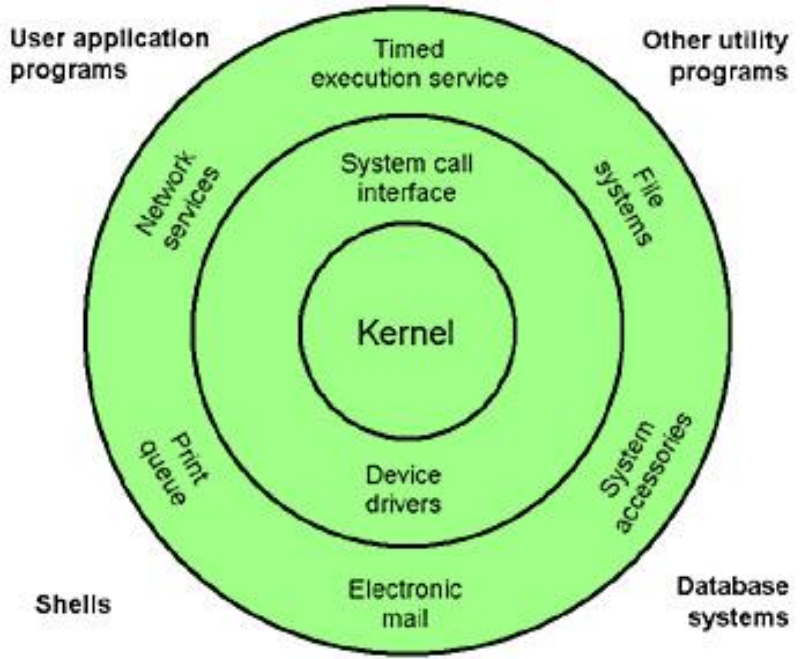


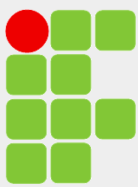
Estrutura Básica de um SO



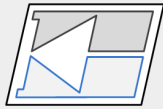
Um SO fornece:

- Alocação de recursos.
- Gerenciamento de tarefas.
- Abstração do HW.
- Interfaces de I/O.
- Controle de tempo.
- Tratamento de erros.





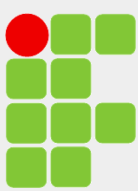
Classificação



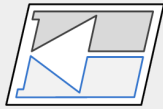
Depende:

- tempo de resposta:
 - batch/ interativo/ tempo-real
- usuário:
 - mono/multiusuário
- modo de execução:
 - mono/ multitarefa
- outros:
 - sistema embarcado, portátil, padronizado

Batch processing is execution of a series of programs on a computer without manual intervention.



Fluxo de Programa

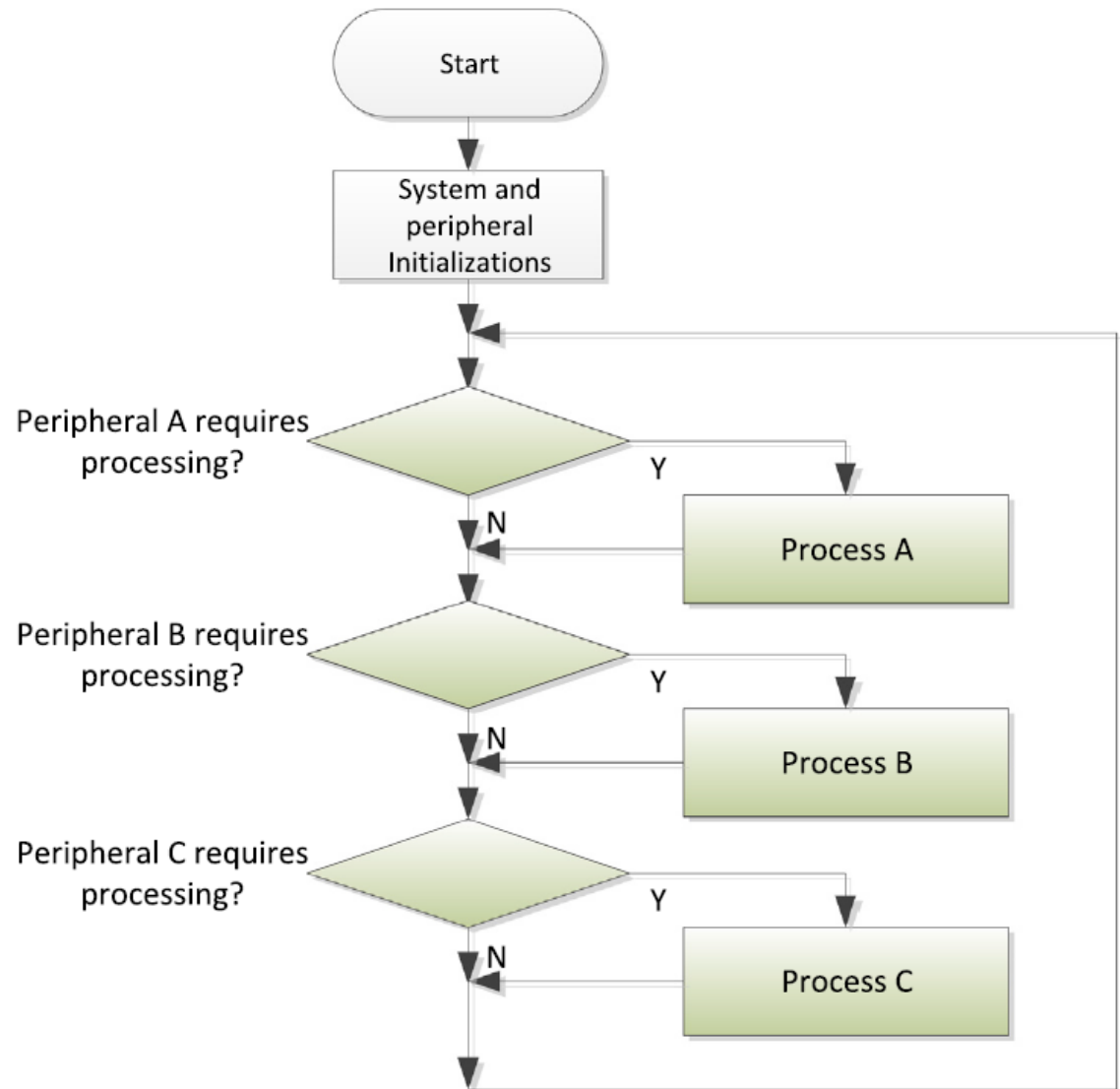


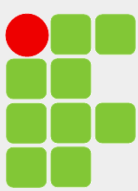
Polling

O processador pode esperar até existir dados para processar.

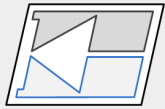
A fluxo do programa é sequencial determinado por um laço infinito -
`while (1) { ... }`

Não é energeticamente eficiente. É difícil de determinar prioridades. É difícil de programar e manter quando a complexidade aumenta.





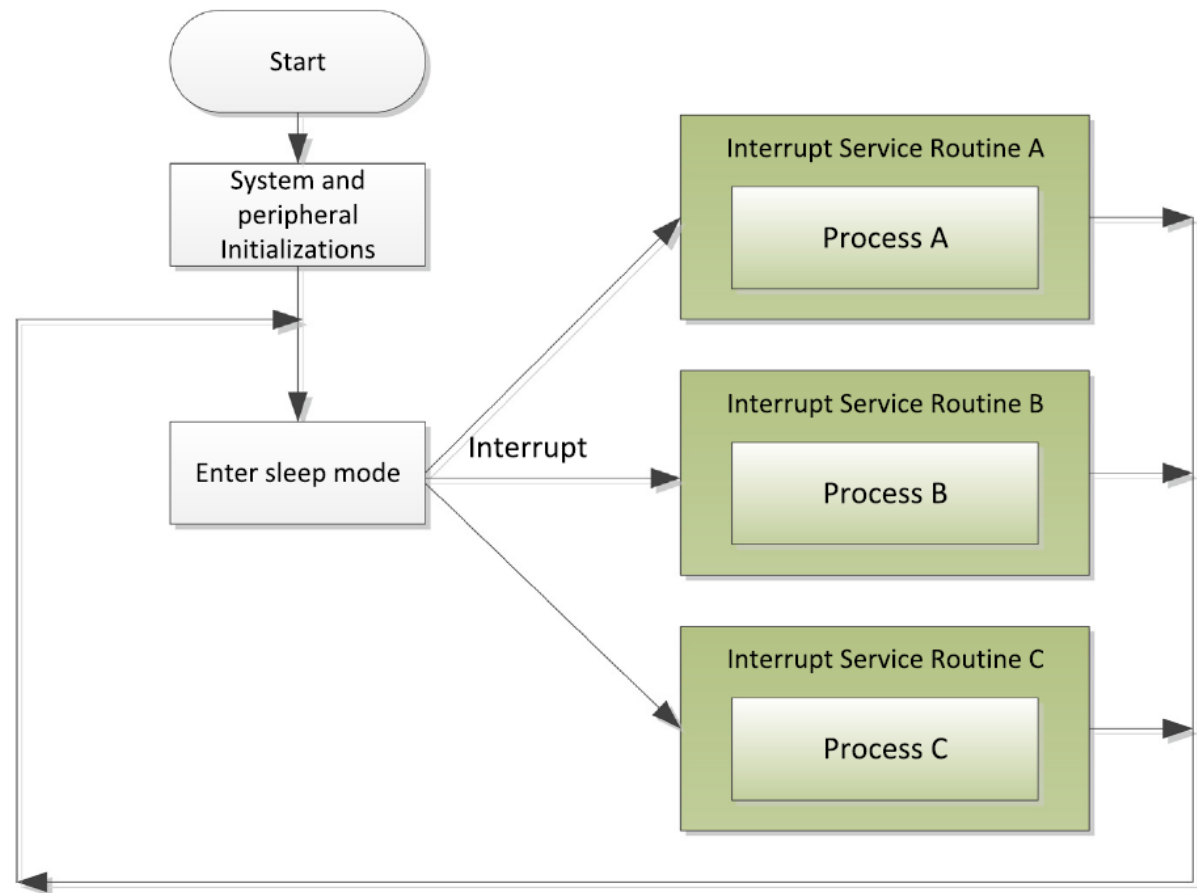
Fluxo de Programa



Interrupt driven

O processador fica “dormindo” e algum periférico o “acorda” quando deseja serviço.

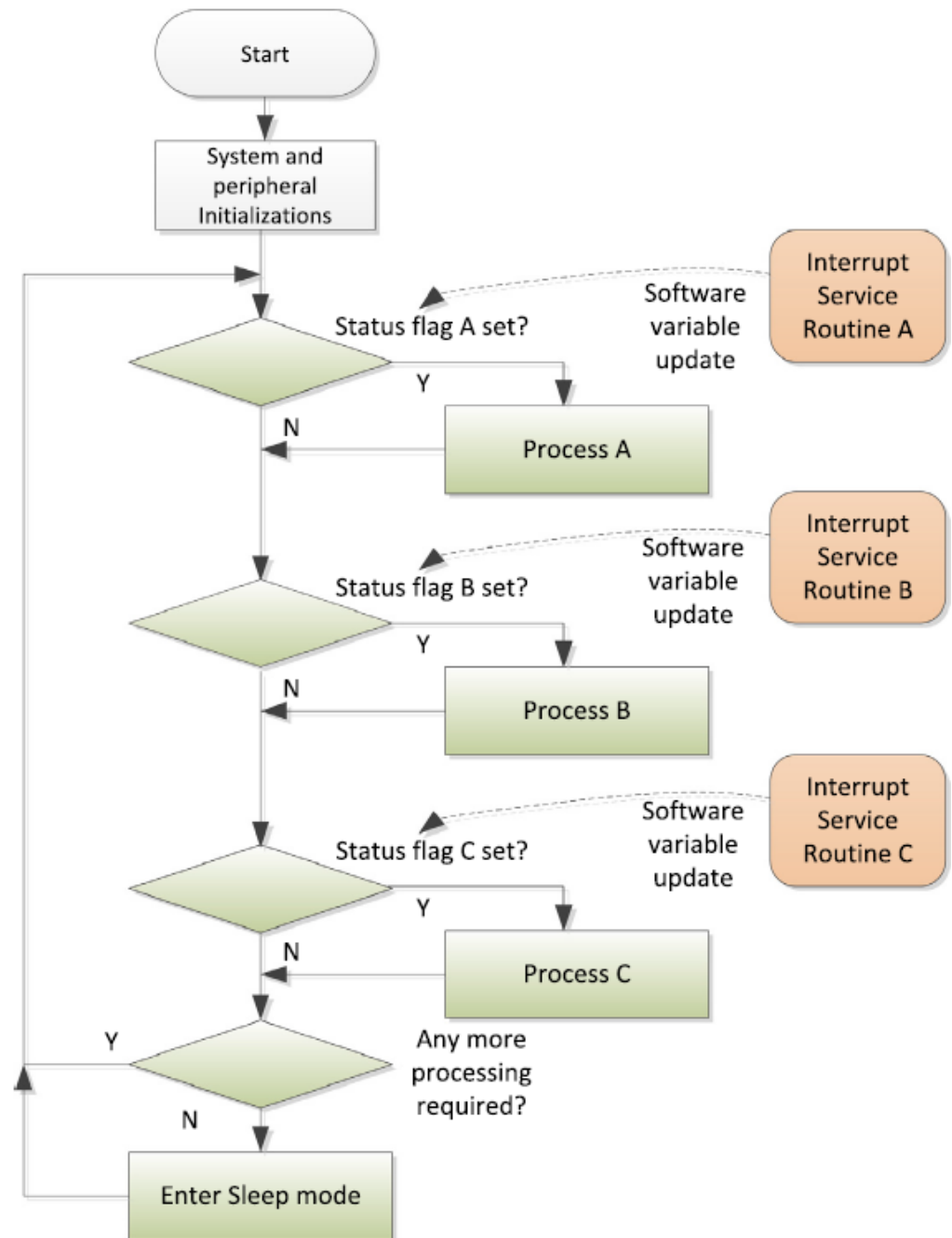
Pode ser dada prioridade no atendimento das interrupções.

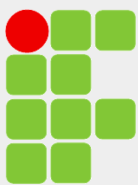


Polling+Interrupt

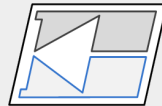
O processamento pode ser particionado em duas partes: uma executada rapidamente e outra mais tarde.

Após a parte rápida executar ela habilita um *flag* para a execução sequencial do trecho de código que não requer velocidade.





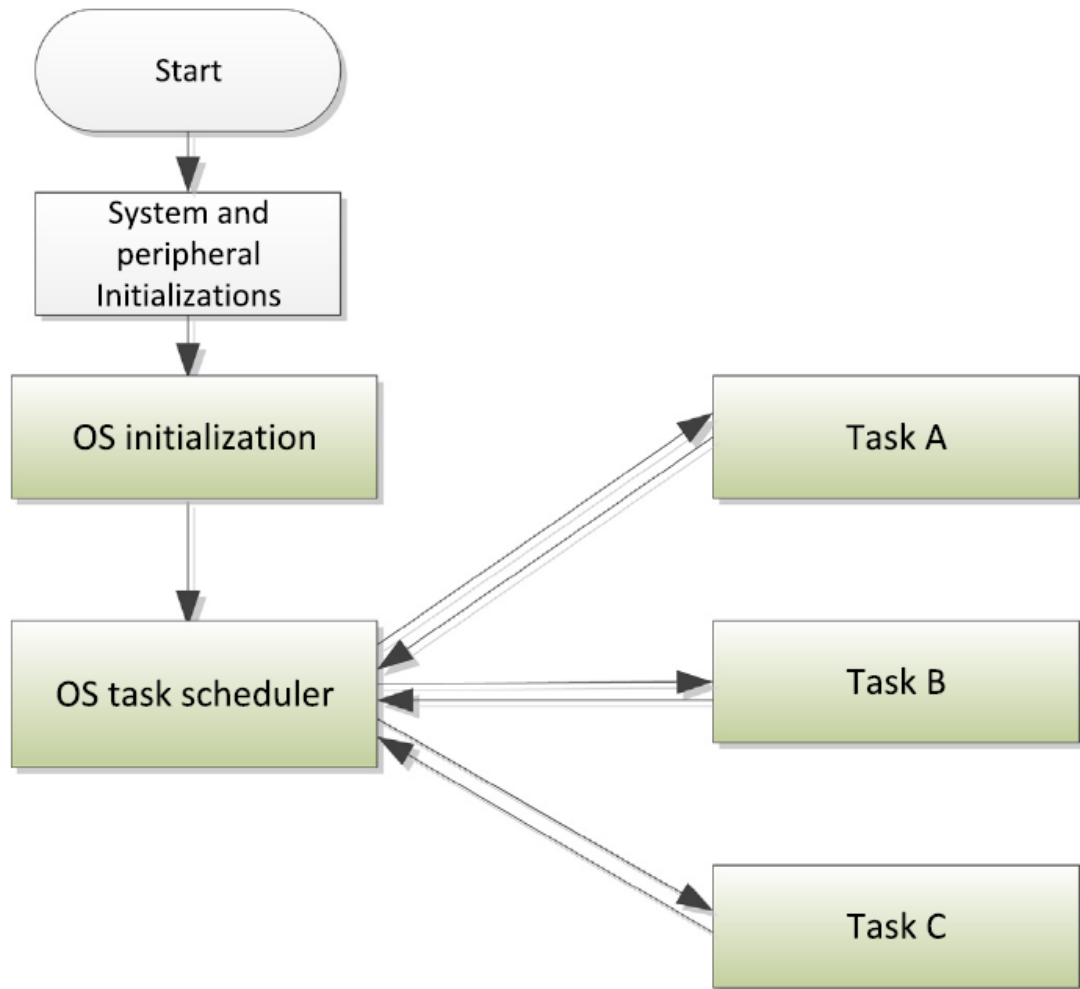
Fluxo de Programa

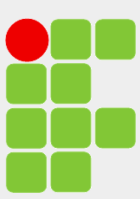


Multi-tasking

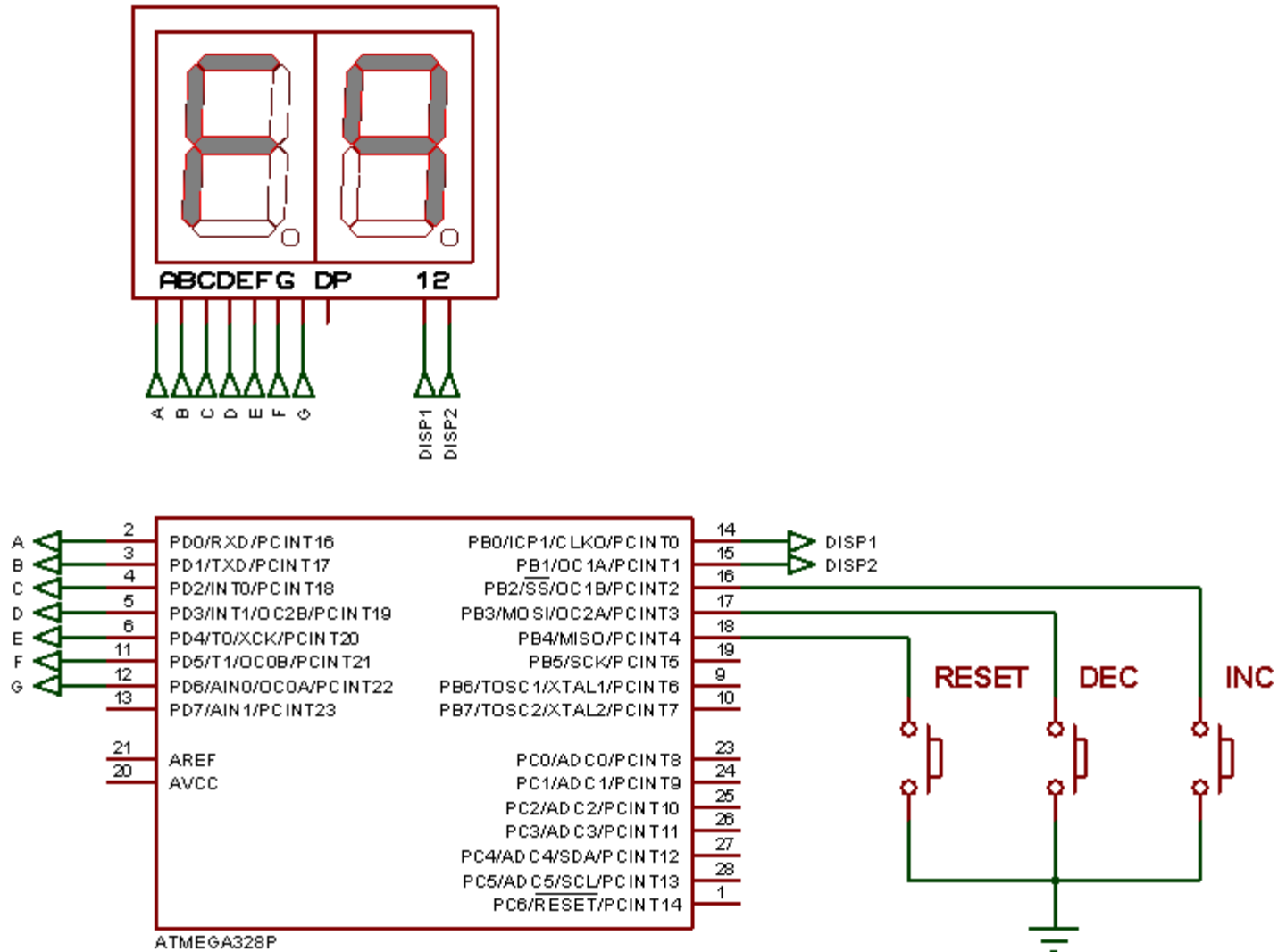
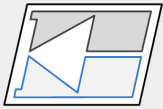
Empregado em aplicações complexas, onde tarefas longas precisam ser divididas e executadas concorrentemente.

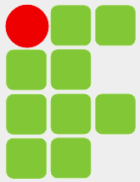
Nesse caso, é necessário um RTOS, que dividirá o tempo de processamento de cada tarefa e gerenciará prioridades e intercomunicações.



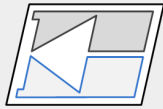


Laço infinito x RTOS





Laço infinito x RTOS



```
int main()
{
    Inicializacoes_normais();

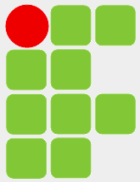
    while(1)
    {
        Multiplexa_Display();
        Incrementa_Contagem();
        Decrementa_Contagem();
        Inicializa_Contagem();
    }

    //-----
    //funções do programa
    //-----
    void Multiplexa_Display() {.....};
    void Incrementa_Contagem() {.....};
    void Decrementa_Contagem() {.....};
    void Inicializa_Contagem() {.....};
}
```

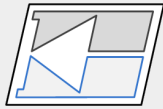
```
int main()
{
    Inicializacoes_normais();
    Inicializacoes_RTOS();//cria tarefas, etc

    while(1);

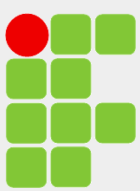
    //-----
    //tarefas do RTOS
    //-----
    void Multiplexa_Display() {while(1){...}};
    void Incrementa_Contagem() {while(1){...}};
    void Decrementa_Contagem() {while(1){...}};
    void Inicializa_Contagem() {while(1){...}};
}
```



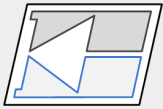
Laço Infinito x RTOS



Laço Infinito	RTOS
Pode utilizar funções ou não. Cada função não pode conter um laço infinito.	Cada função é uma tarefa independente do RTOS, com um laço infinito de execução.
As tarefas são sequenciais, executadas no programa principal dentro de um laço infinito. O paralelismo que se consegue é com o uso das interrupções do microcontrolador.	As tarefas são concorrente. O RTOS gerencia sua execução. Existe a sensação de paralelismo na execução. As interrupções normais do microcontrolador ainda podem ser utilizadas.
O programa pode travar em alguma função, como por exemplo, quando existe um laço de espera que nunca é finalizado, pois a execução do programa é sequencial.	O programa não trava em funções com laços de espera. Se uma tarefa ficar presa, o RTOS vai passar o controle para outra função com o passar do tempo.
Dependendo do problema, maior complexidade de programação.	Menor complexidade de programação – melhor estruturação.
Uso de menor quantidade de memória.	Emprega mais memória, impacto maior na RAM.
Pode obter o máximo de desempenho, com a utilização constante da CPU para a execução do programa.	Perda de processamento na troca de contexto pela CPU (overhead).



RTOS



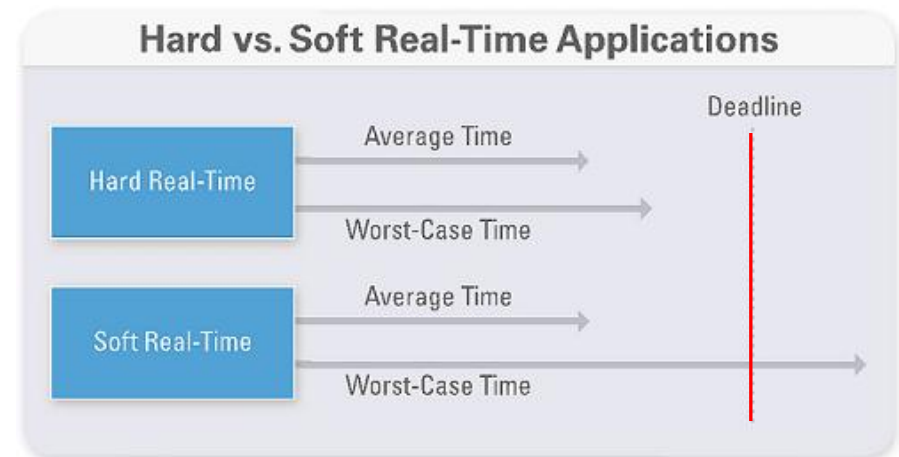
Um Sistema Operacional de Tempo Real é um software que gerencia o tempo de um μ controlador para garantir que eventos temporais críticos sejam processados da forma mais eficiente possível.

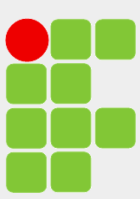
O uso de um RTOS simplifica a programação pois permite que elementos sejam divididos independentemente em tarefas.



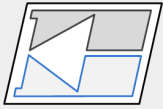
Hard Real-Time System: a ação deve ocorrer absolutamente em certo intervalo de tempo (o tempo de ação é crítico).

Soft-Real-Time System: perder ocasionalmente uma certa ação, não é desejável, mas aceito, desde que não cause nenhum dano permanente.

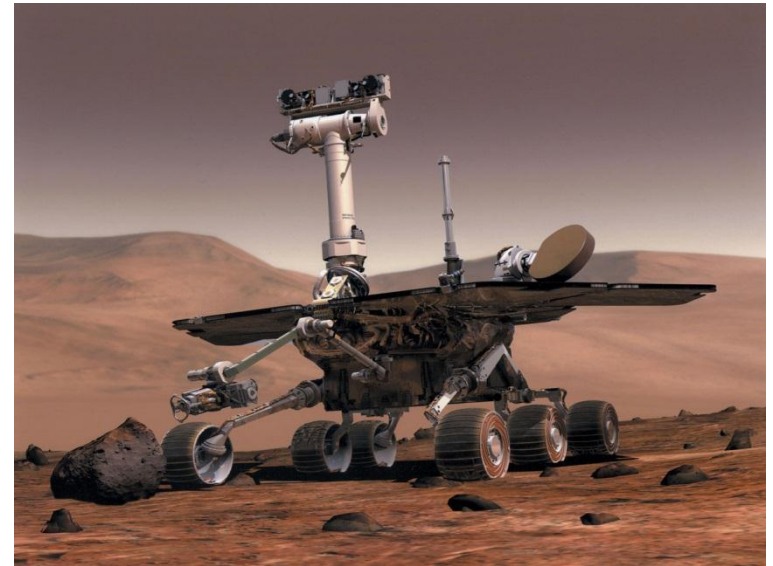
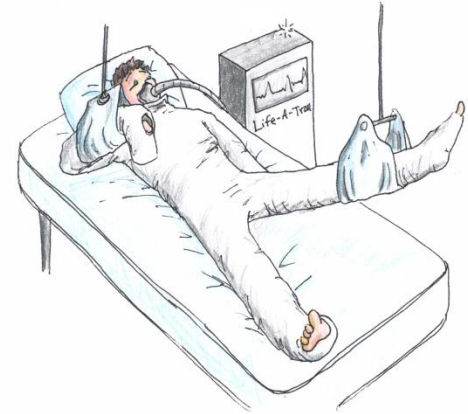


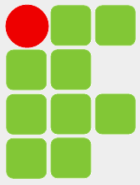


RTOS

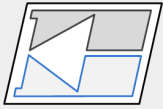


Em um RTOS somente o programa do projetistas é executado, o usuário não pode fazer o seu programa, o que torna o sistema mais seguro.

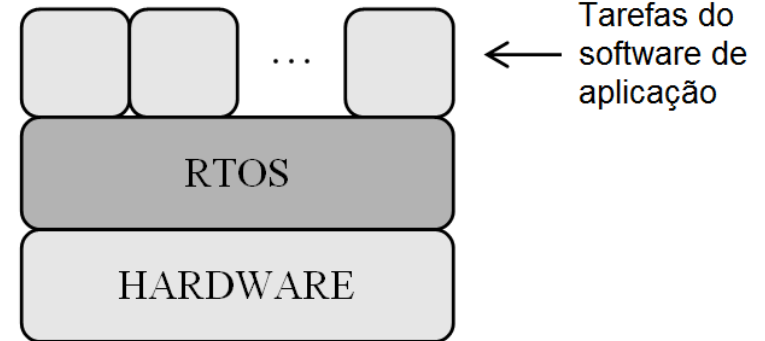




RTOS

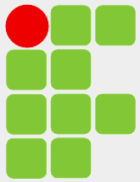


O *kernel* de um RTOS fornece uma camada de abstração para esconder detalhes do hardware do processador sob o qual o software de aplicação será executado, fornece vários serviços:

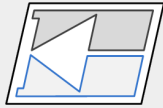


Permite que múltiplos processos sejam executados concorrentemente dividindo o tempo do processador em *time slots* e alocando um *slot* para cada tarefa. Usa um temporizador para gerar interrupções a intervalos fixos (*ticks*) acionando o **gerenciador de tarefas** que faz a troca de contexto suspendendo a tarefa em execução e começando outra, conforme a prioridade.

Além do gerenciador de tarefas, possui outros serviços para a comunicação entre tarefas e divisão dos recursos do HW.



Tarefas

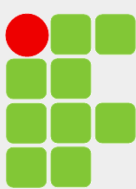


Uma tarefa (*task*) é um programa que pensa que ele tem o processador inteiro para ele. Cada tarefa é definida por uma função no programa e tem sua própria pilha. O núcleo do RTOS se responsabiliza pelo ajuste das pilhas e pelo gerenciamento das tarefas de acordo com a prioridade dada.

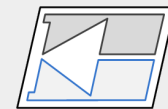
Exemplos de tarefas

- Leitura de sensores
- Cálculos
- Laços de controle
- Atualização de saídas
- Leitura de teclados
- Escrita em displays
- Comunicações Seriais
- *Data logging*
- Monitoramento

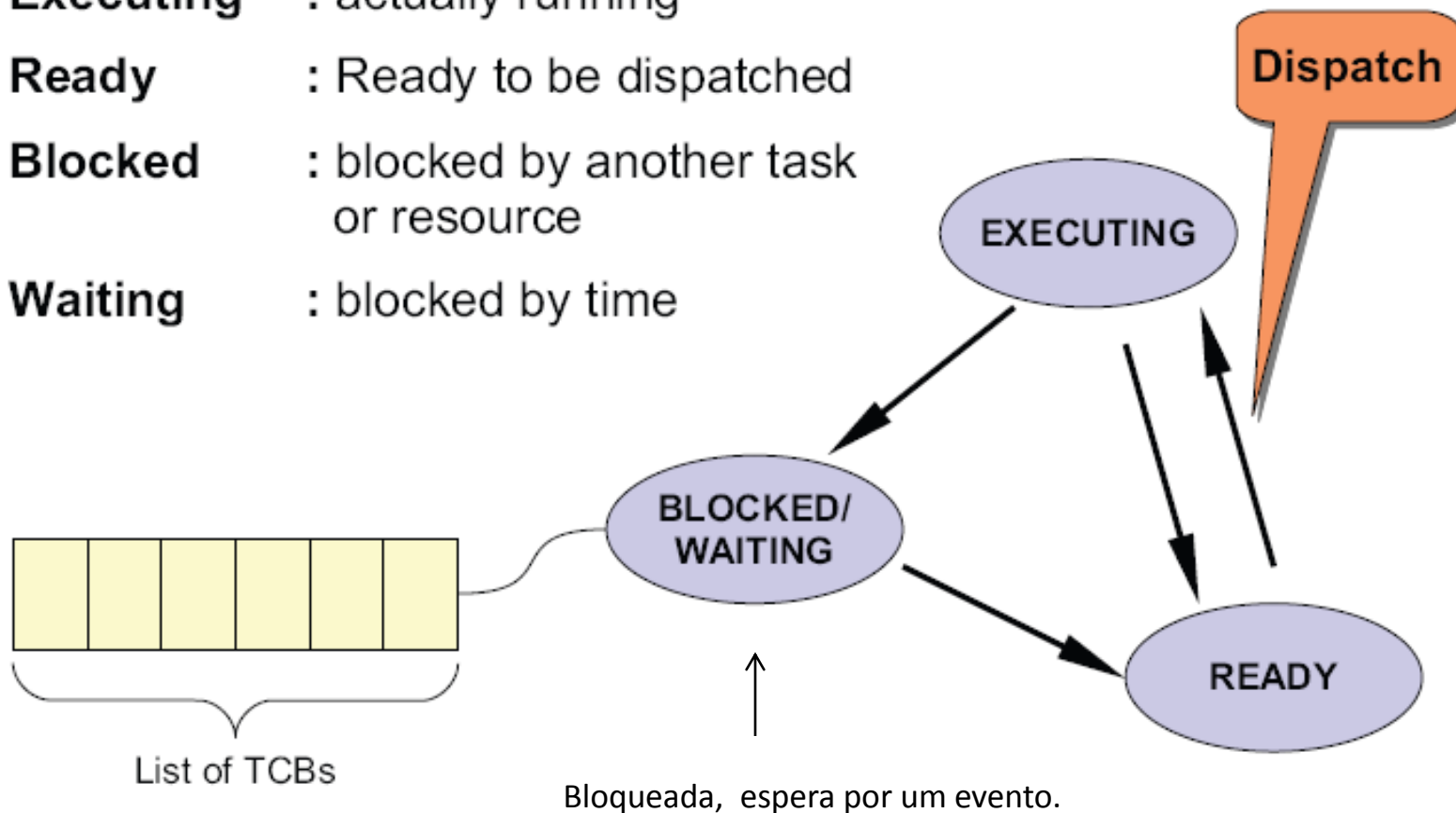
```
void my_Task()  
{  
    while(1)  
    {  
        //faz alguma coisa(seu código)  
        //espera por um evento  
        //atraso de n clock ticks  
        //espera pelo semáforo  
        //espera por uma mensagem  
        //outra coisa  
        //faz alguma coisa(seu código)  
    }  
}
```

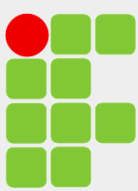


Estados de uma Tarefa

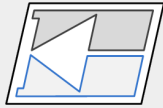


- **Executing** : actually running
- **Ready** : Ready to be dispatched
- **Blocked** : blocked by another task or resource
- **Waiting** : blocked by time





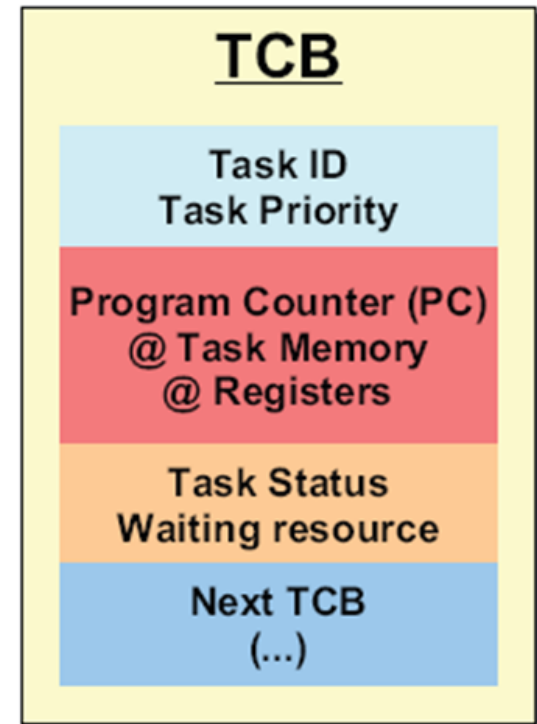
Task Control Block

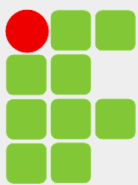


O programador deve fornecer informações para o *kernel* sobre a tarefa quando ela é criada (como prioridade e espaço de pilha). O *kernel* mantém uma estrutura de dados para cada tarefa criada chamada de Bloco de Controle de Tarefas (TCB).

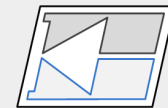
O TCB contém:

- A identificação da tarefa.
- A prioridade da tarefa.
- O estado (pronta ou esperando para ser executada).
- Cópia do último contexto (topo da pilha, registradores, ...).
- Outras informações.

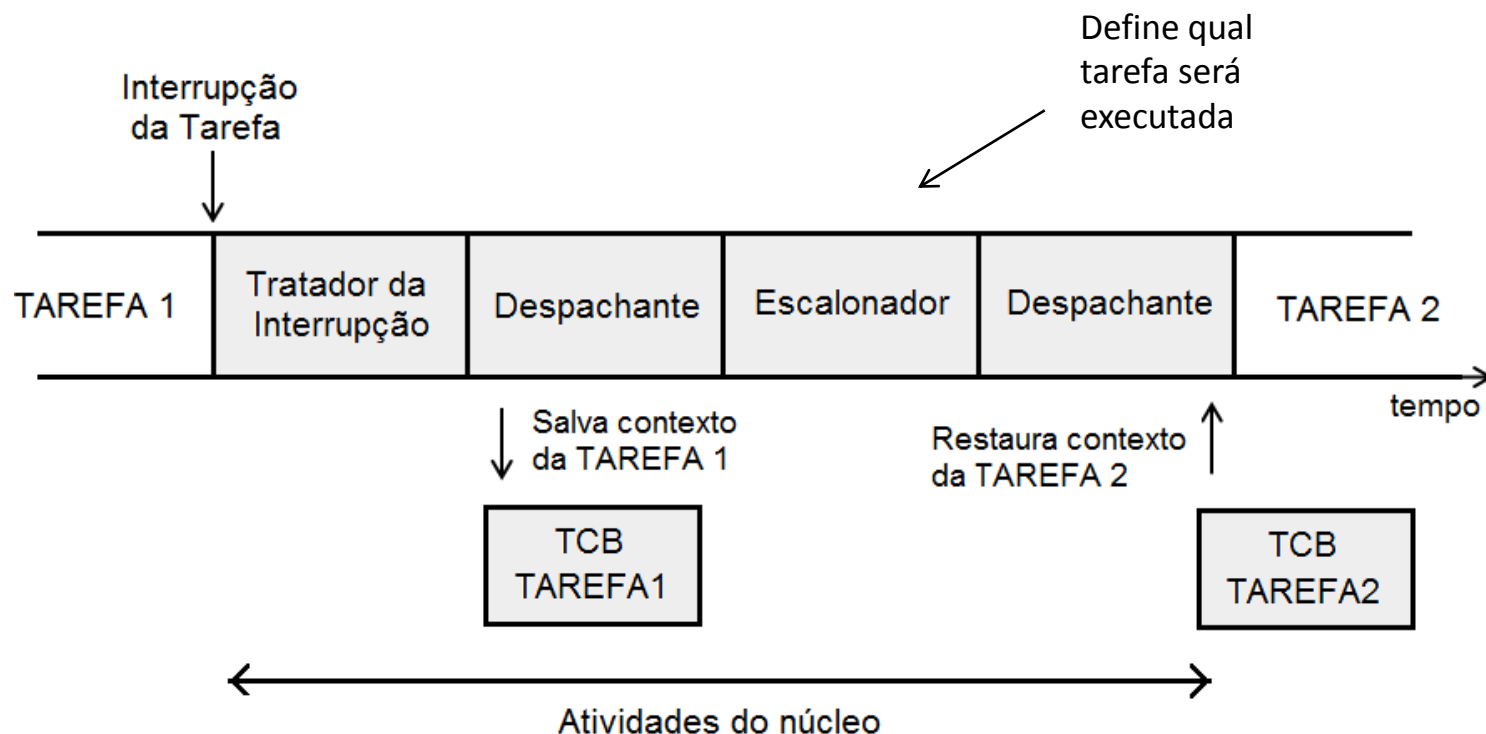




Task Control Block

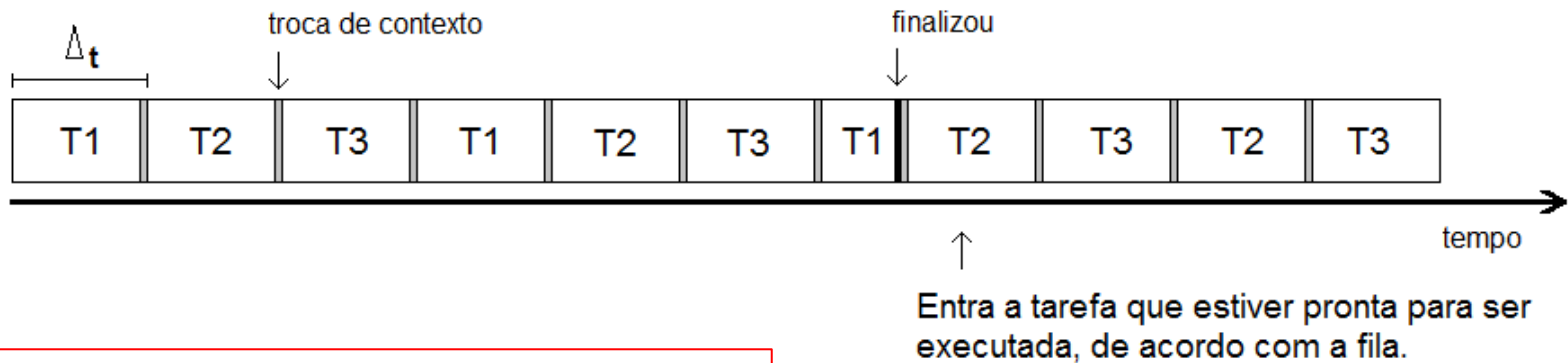


Sempre que ocorre uma troca de tarefas, o contexto da tarefa em execução (PC, pilha e registradores) é salvo pelo núcleo do SO no TCB, para que ela possa recomeçar de onde parou na próxima chamada. Por sua vez, a tarefa que será executada tem seu contexto restaurado a partir do seu TCB antes de receber processamento.



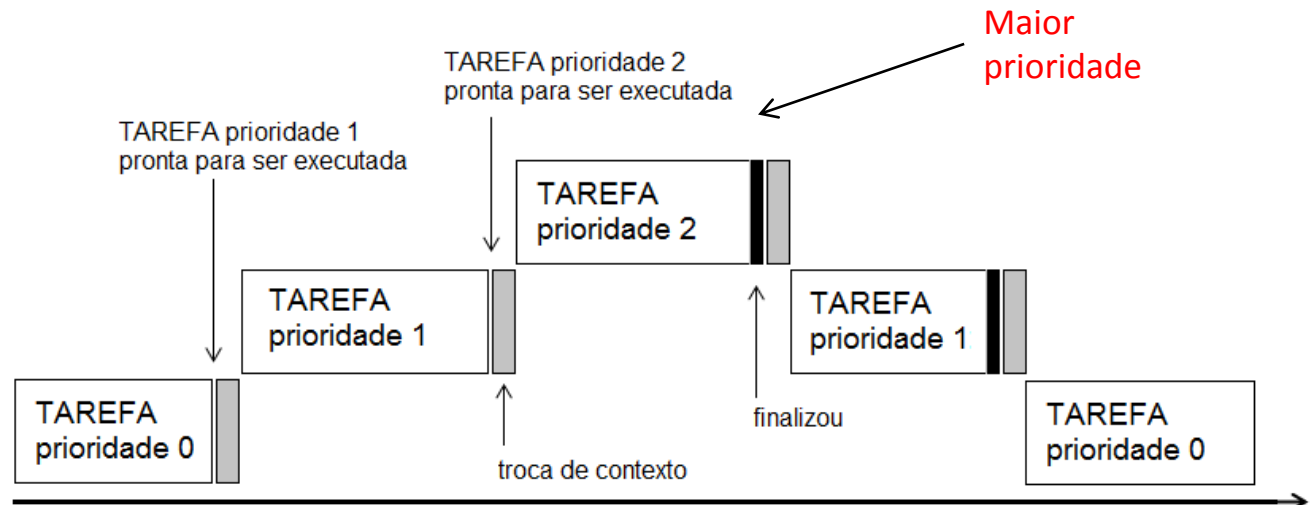
Prioridade: modos de execução

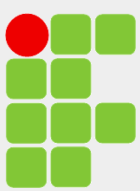
Round-robin: todas as tarefas possuem a mesma prioridade e são organizadas em fila para execução sequencial, com tempo fixo para cada tarefa (*clock tick*).



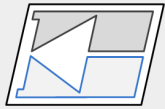
Preemptivo: executa a tarefa de mais alta prioridade sempre que estiver pronta.

Tarefas de igual prioridade serão organizadas de modo *round-robin* e tarefas de maior prioridade irão preceder tarefas de menor prioridade entrando no estado de execução sob demanda.

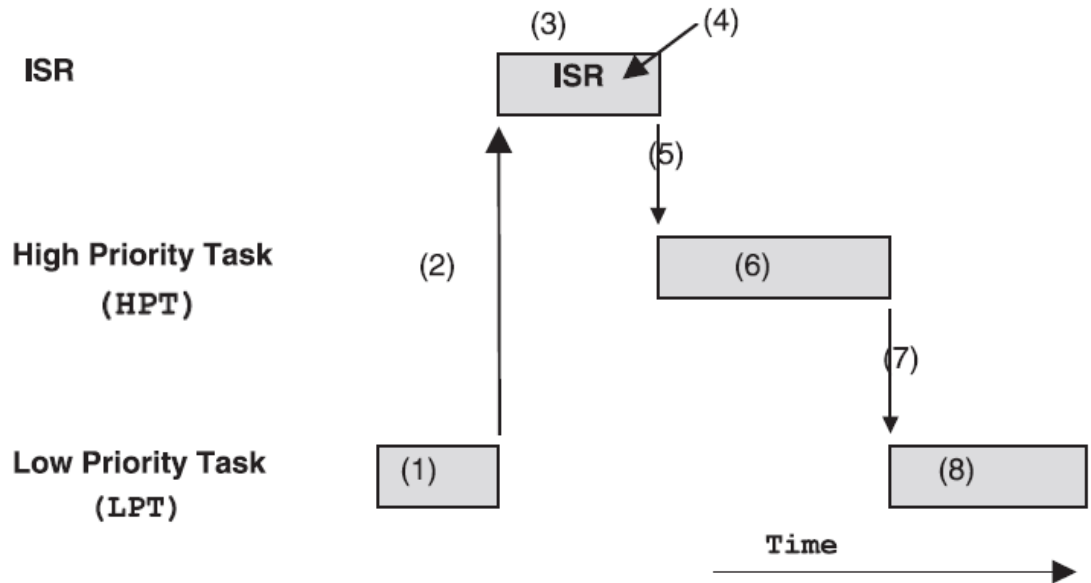




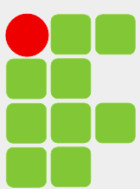
RTOS Preemptivo



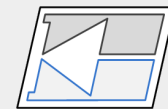
- 1 – Executando uma tarefa de baixa prioridade.
- 2 – Evento da alta prioridade ocorreu interrompendo LPT.
- 3 – O contexto da LPT é salvo e é executada a ISR.
- 4 – A ISR chama um serviço do *Kernel* para acordar a tarefa de alta prioridade e deixa-la pronta para execução.



- 5 – Após completa, a ISR invoca mais uma vez o *kernel* que volta agora para a HPT ao invés da LPT.
- 6 – a HPT executa até completar, ou se desejado, espera por um evento.
- 7 – Se a HPT esperar por um evento, ela é suspensa e a LPT volta de onde parou, carrega o contexto da LPT.
- 8 – A LPT continua a executar.



RTOS Preemptivo



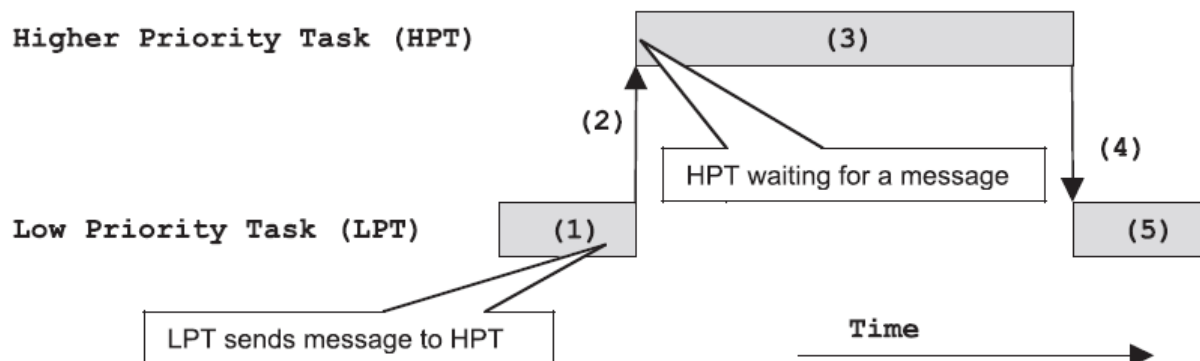
1 – LPT está executando e envia uma mensagem para HPT.

2 – Como a HPT tem maior prioridade, o *kernel* suspende a LPT (salva contexto) e retoma (restaura contexto) HPT (que espera pela mensagem).

3 - a HPT executa até completar ou ser interrompida

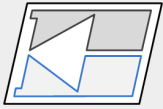
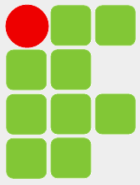
4 – Se a HPT deseja outra mensagem da LPT, ela é suspensão e LPT é retomada.

5 – A LPT continua a execução.



Se todas as tarefas no sistema estão esperando por eventos. O *kernel* executa (até que o evento ocorra) uma tarefa chamada *Idle Task*, que basicamente é um laço infinito. Se o HW funciona com baterias o processador pode ser colocado em algum modo de economia de energia.

```
void OS_Idle_Task()  
{  
    while(1)  
    {  
        //place CPU in LOW POWER sleep mode  
        //or other stuff  
    }  
}
```

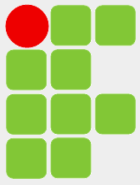
The Clock Tick

Todo RTOS necessita de um fonte de tempo para gerar interrupções periódicas e atrasos, comumente denominada *clock tick*. Geralmente essa fonte é provida por um temporizador do μ controlador que interrompe a CPU a cada 10 a 100 ms. O *clock tick* é usado na divisão de tempo entre as tarefas.

Tarefas podem ser bloqueada por um determinado número de *clock ticks*. Isso pode ser importante para que tarefas de alta prioridade permitam que tarefas de baixa prioridade executem.

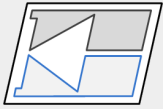


```
void my_Task ()
{
    while (1)
    {
        OS_Time_Delay (10) ;
        /*suspende a tarefa corrente
           por 10 ticks*/
        // código da tarefa
    }
}
```



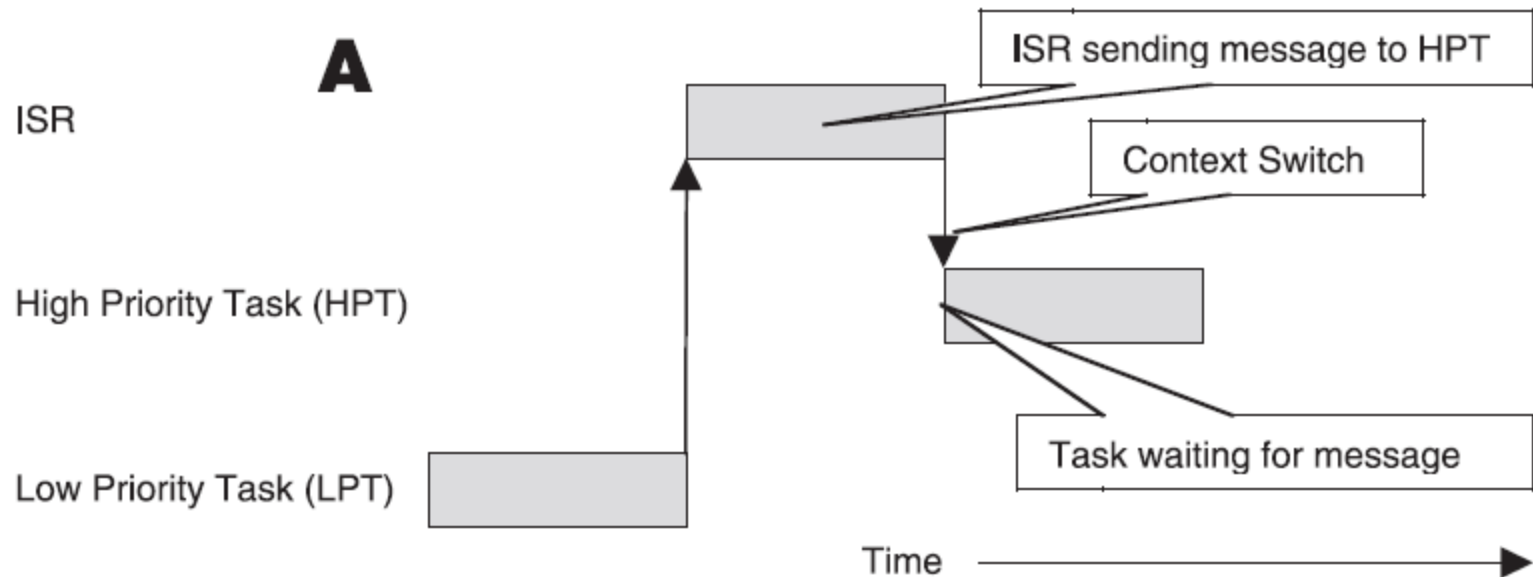
Gerenciador de Tarefas

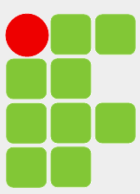
(Escalonador – *Task Scheduling*)



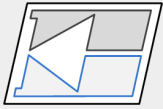
É uma função do *kernel* para determinar se uma tarefa mais importante necessita execução (define qual tarefa será executada). Ocorre nas seguintes condições:

1 – Uma rotina de interrupção (ISR) envia uma mensagem para uma tarefa.

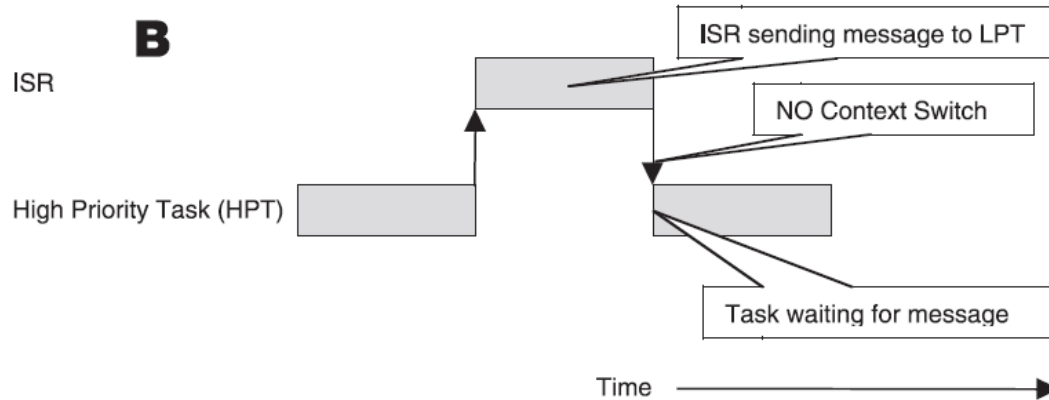




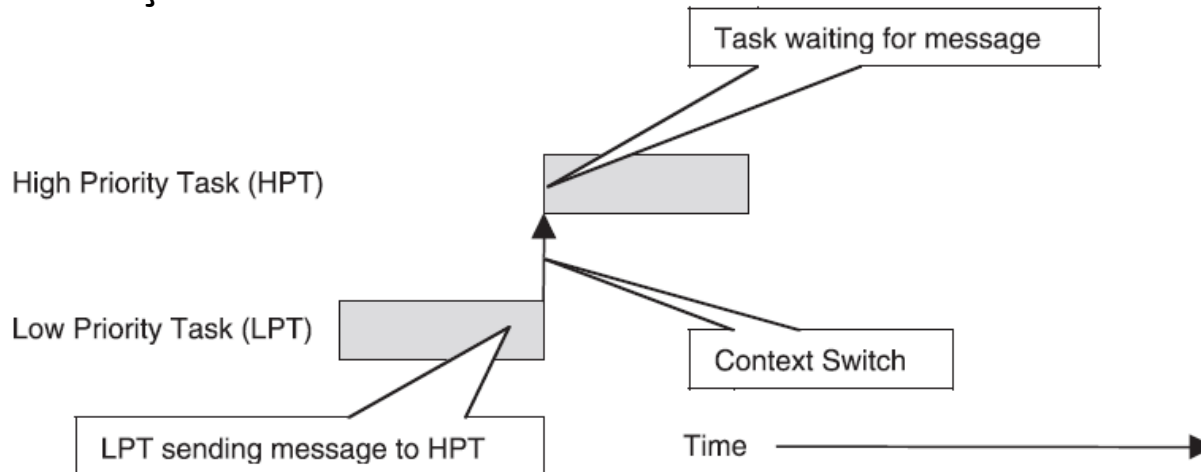
Task Scheduling

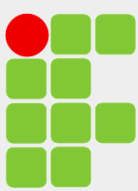


1 –

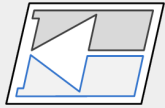


2 – Uma tarefa envia uma mensagem para outra tarefa. Se a tarefa que espera for mais importante, ela será executada, senão a tarefa que envia a mensagem continua a execução.



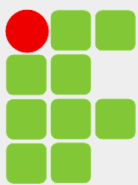


Task Scheduling

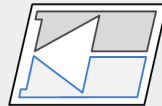


3 – Uma tarefa deseja esperar um tempo para execução - `OS_Time_Delay()`. Nesse caso, a tarefa é suspensa, e o *kernel* seleciona e executa a próxima tarefa mais importante que esteja pronta para execução.

Para que uma tarefa seja executada ela deve estar pronta para execução. Ou seja, se ela estiver esperando por um evento e ele ocorrer, o *kernel* fará a tarefa elegível. O escalonador só executará tarefas prontas para execução.



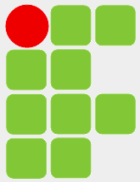
Kernel Services



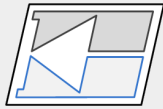
Até agora foram vistos os seguintes pontos sobre um RTOS:

- Divisão da aplicação em várias tarefas.
- Requer uma fonte de *clock (tick)* para suspender tarefas, determinar o tempo de execução ou limitação de tempo para a execução de tarefas, bem como para o recebimento de mensagens.
- Uma tarefa ou ISR pode enviar mensagens para outras tarefas.
- Um RTOS preemptivo sempre tenta executar as tarefas de maior prioridade que estão prontas para execução.
- A troca de contexto é empregada para salvar o estado corrente de uma tarefa e a restauração de contexto é empregada para carregar os dados para a retomada de uma tarefa.

Existem muitos outros serviços providos por RTOS comerciais, alguns serão vistos a seguir.



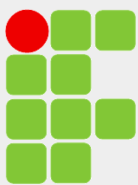
Comunicação entre Tarefas e Sincronização



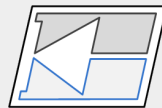
Em um RTOS preemptivo são necessários mecanismos para a comunicação e a sincronização entre tarefas, porque na ausência deles, as tarefas podem transmitir informações corrompidas ou interferirem umas nas outras.

Por exemplo, uma tarefa pode perder o processador quando está no meio da atualização de uma tabela de dados. Se uma segunda tarefa, que adquiriu o processador, ler a partir dessa tabela, ela lerá uma combinação de algumas áreas de dados recém atualizadas e de outras que ainda não foram atualizadas. Essa atualização parcial dos dados deve torná-los inconsistentes.

- Tasks need to communicate
 - sharing data
 - i.e. producer / consumer
 - mutual exclusion (mutex)
- Tasks need to synchronize
 - “meeting points”
 - i.e. wait for an intermediate result
 - i.e. wait for the end of another task



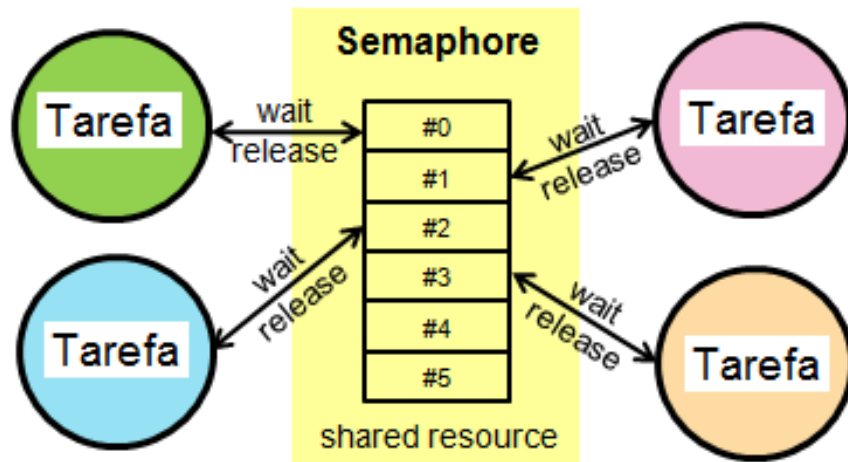
Semáforo

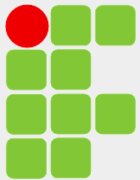


Usado para garantir acesso exclusivo a recursos comuns, tais como: variáveis compartilhadas, matrizes, estrutura de dados e dispositivos de I/O.

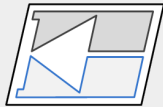
Se o semáforo está disponível, a tarefa que deseja o recurso pode utilizá-lo, caso contrário, ela fica bloqueada e entra para a lista de espera.

Uma vez que a tarefa que possui o recurso o libera, o *kernel* verifica se existe alguma tarefa esperando pelo recurso, passando esse para a tarefa de maior prioridade. Se uma tarefa de maior prioridade estava com o recurso, ele vai continuar com ela até que ela não mais o deseje.



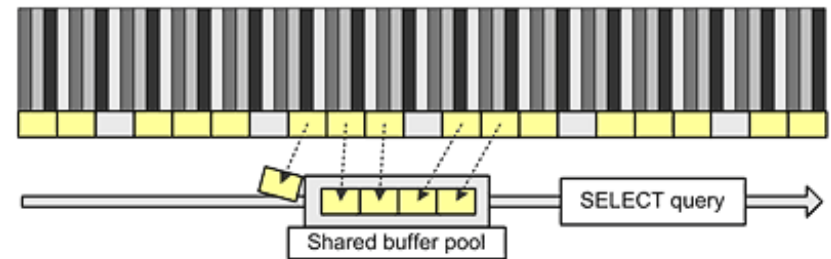


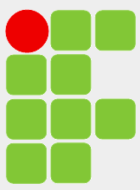
Semáforo



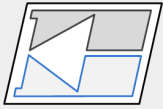
Existem 3 tipos de semáforos:

- Binário é usado para acessar um única fonte (ex. só tem uma UART).
- *Counting* é usado para acessar alguma de várias fontes idênticas (ex. *buffer pool*).
- *Mutual Exclusion Semaphore* – MUTEX é um tipo especial de semáforo binário (com “inteligência”) empregado para reduzir um problema chamado inversão de prioridade, comuns aos outros tipos de semáforos.





Exemplo



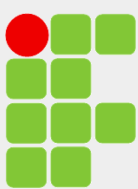
```
semaphore s =  
    init_semaphore(0);
```

Task A

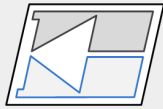
```
void taskA() {  
    while (1) {  
        ...  
        produce_result();  
        signal_semaphore(s);  
        ...  
    }  
}
```

Task B

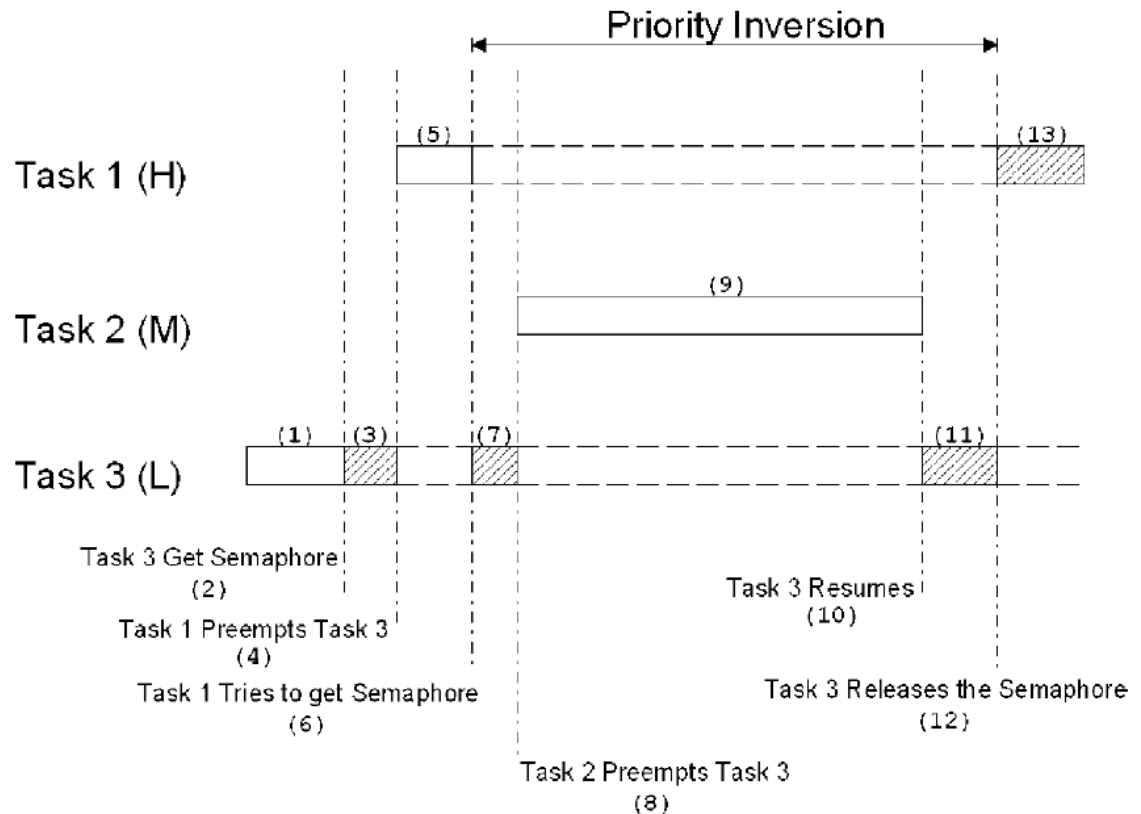
```
void taskB() {  
    while (1) {  
        ...  
        wait_semaphore(s)  
        get_result();  
        ...  
    }  
}
```



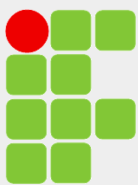
Inversão de Prioridade



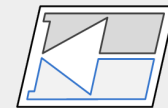
- 1 - T1 e T2 esperam por um evento e T3 está executando.
- 2 - Em algum momento T3 obtém um semáforo.
- 3 - T3 opera com o recurso obtido.
- 4 - O evento esperado pela T1 ocorre e ele precede T3.
- 5 - T1 executando
- 6 - T1 tenta obter o semáforo que esta com T3 (entra para a lista de espera).
- 7 - T3 volta a executar.
- 8 - T3 é precedida por T2 que esperava um evento.
- 9 - T2 está executando.
- 10 - T2 termina e a CPU volta a T3.
- 11 - T3 está executando.
- 12 - T3 libera o semáforo.
- 13 - T1 que espera pelo recurso obtém o processamento.



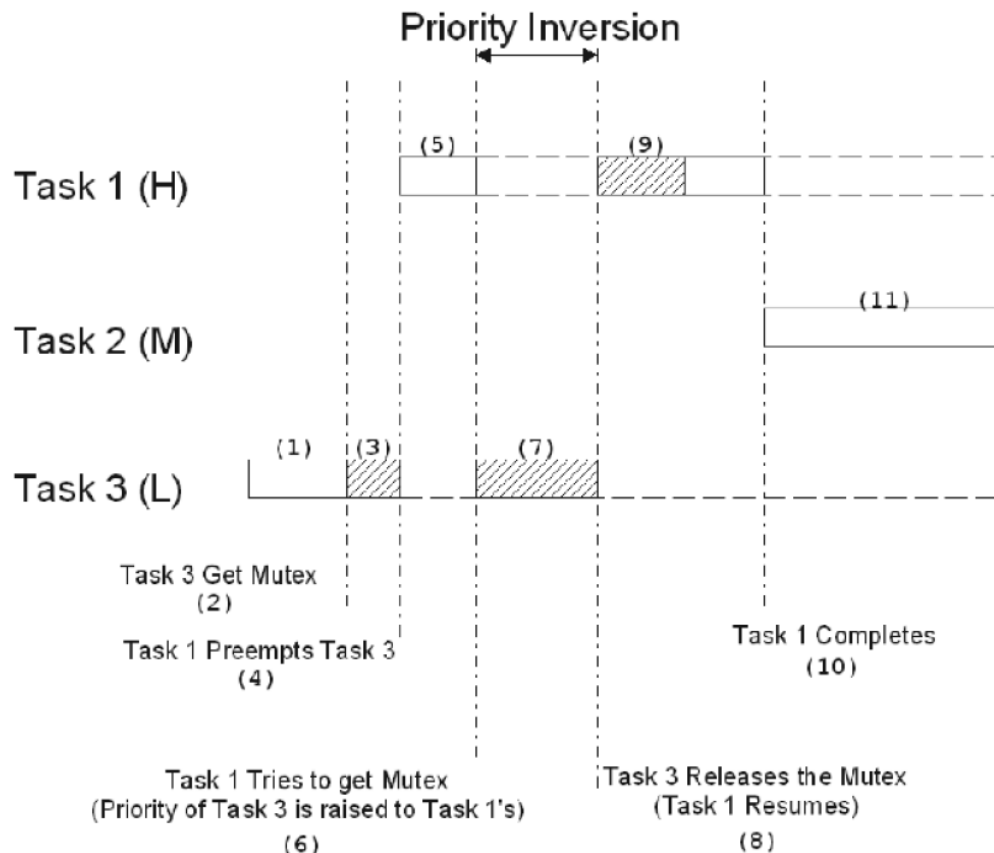
A prioridade da T1 foi virtualmente reduzida a da T3 porque espera pelo recurso possuído por essa. O cenário piorou quando T2 precedeu T3, atrasando ainda mais T1.



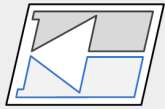
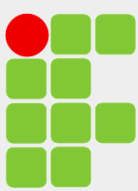
Inversão de Prioridade



- 1 – T3 está executando.
- 2 – T3 obtém o MUTEX.
- 3 – T3 está executando.
- 4 – T3 obtém o recurso e é precedida por T1.
- 5 – T1 está executando.
- 6 – T1 tenta obter o MUTEX, o *kernel* então muda a prioridade de T3 para o mesmo nível de T1.
- 7 – T1 entra para a lista de espera e T3 volta a operar com o recurso.
- 8 – T3 libera o recurso, pela lista de espera o recurso vai para T1. T3 volta a ter sua prioridade original.
- 9 – T1 usa o recurso liberado.
- 10 – T1 termina.
- 11 – T2 que possui média prioridade obtém a CPU.



T2 poderia estar pronta para executar entre os passos 3 e 10 que o resultado seria o mesmo. Ainda existe inversão de prioridade, mas bem menos que no caso anterior.



Deadlock – *Deadly Embrace*



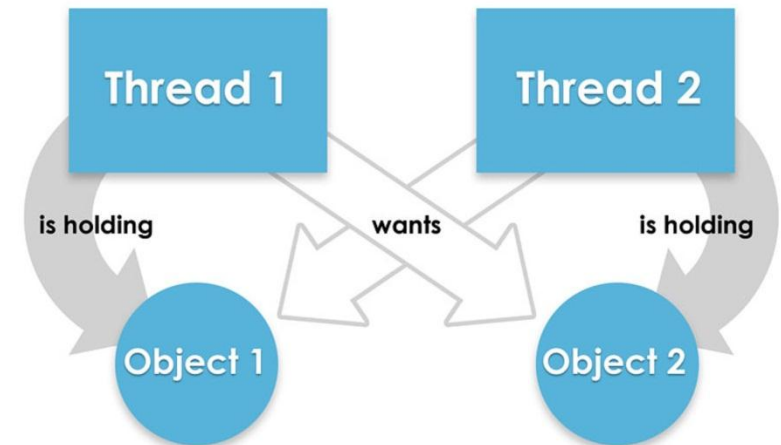
Ocorre quando duas tarefas estão esperando por um recurso que é mantido por elas. Por exemplo, a tarefa A e a tarefa B necessitam do mutex X e Y para poderem executar:

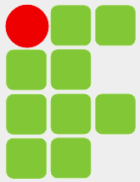
- 1 – A tarefa A executa e obtém o mutex X.
- 2 – A tarefa A é precedida pela tarefa B.
- 3 – A tarefa B obtém o mutex Y antes de tentar obter o mutex X.

Como o mutex X é mantido pela tarefa A, a tarefa B entra no modo bloqueado esperando que o mutex X seja liberado.

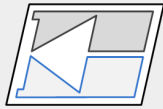
4 – A tarefa A continua a executar (não libera o mutex X), ela tenta obter o mutex Y (que está com a tarefa B), entra no modo bloqueado esperando pelo mutex Y.

Para evitar deadlocks isso deve ser levado em consideração na hora da programação ...

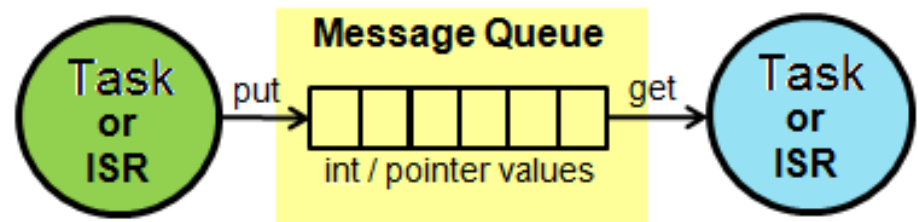
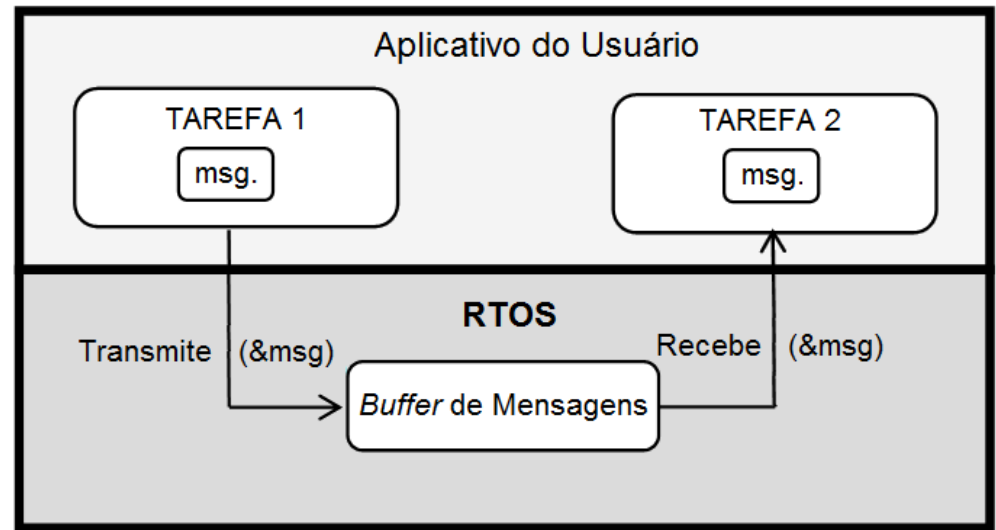


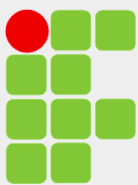


Fila de Mensagens

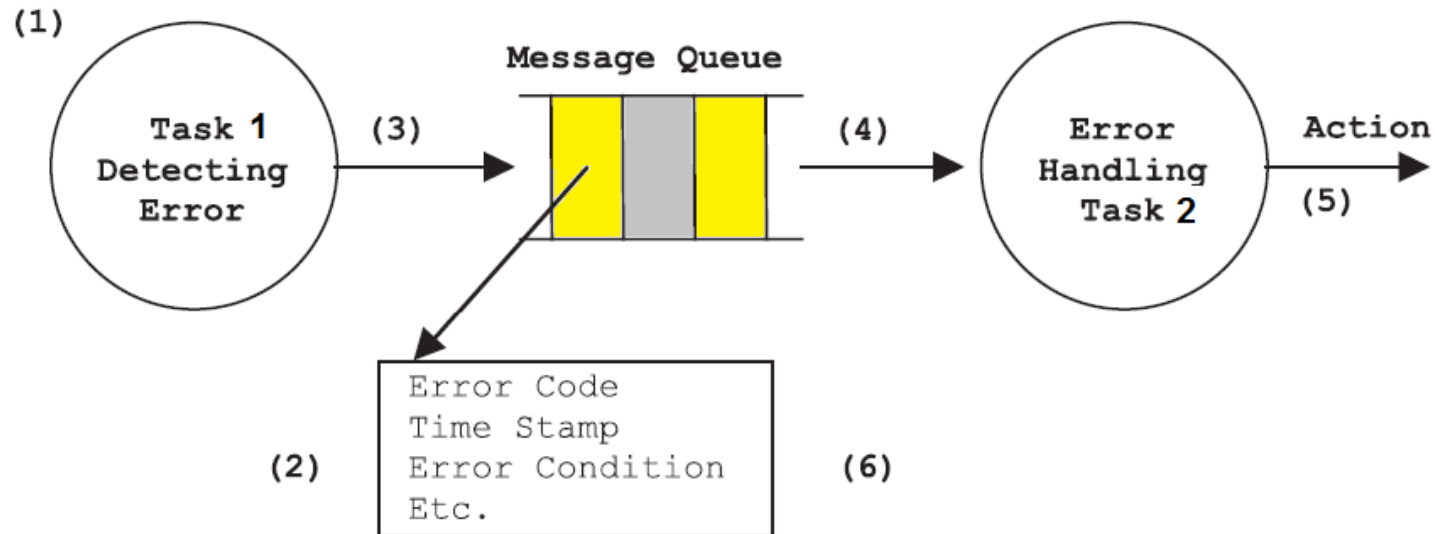
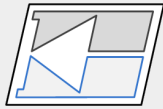


- Uma tarefa ou ISR pode enviar uma mensagem para outra tarefa ou ISR.
- As mensagens são implementadas como *buffers* circulares FIFO. O número de mensagens pode ser configurável.
- O *kernel* fornece um mecanismo para a transmissão das mensagens.
- Geralmente a mensagem é indicada por um ponteiro.

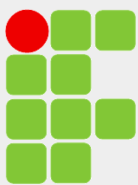




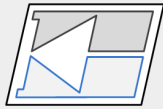
Exemplo



- 1 – T1 detecta uma condição de erro (temperatura muito alta, pressão do óleo muito baixa, ...).
- 2 – T1 aloca um *buffer* para armazenagem das informações sobre o erro.
- 3 – T1 invoca o serviço do *kernel* responsável para passar a mensagem para T2. O *kernel* determina se há alguma tarefa esperando pela mensagem, então a transmite.
- 4 – T2 recebe a mensagem e a trata.
- 5 – T2 pode executar alguma ação: ligar um LED, um relé, escrever a informação em um arquivo.
- 6 – Após terminar, T2 desaloca o *buffer* utilizado.

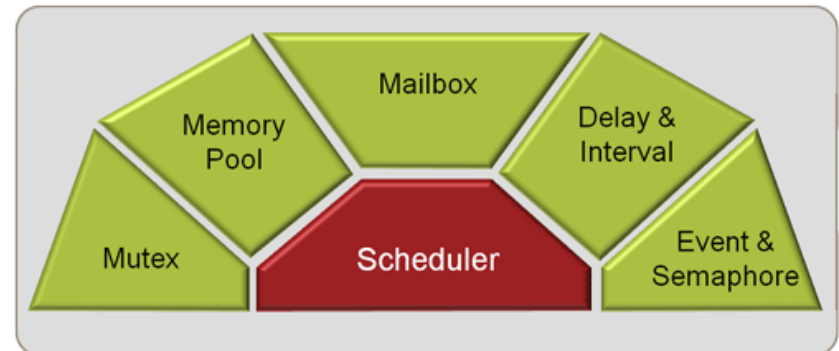
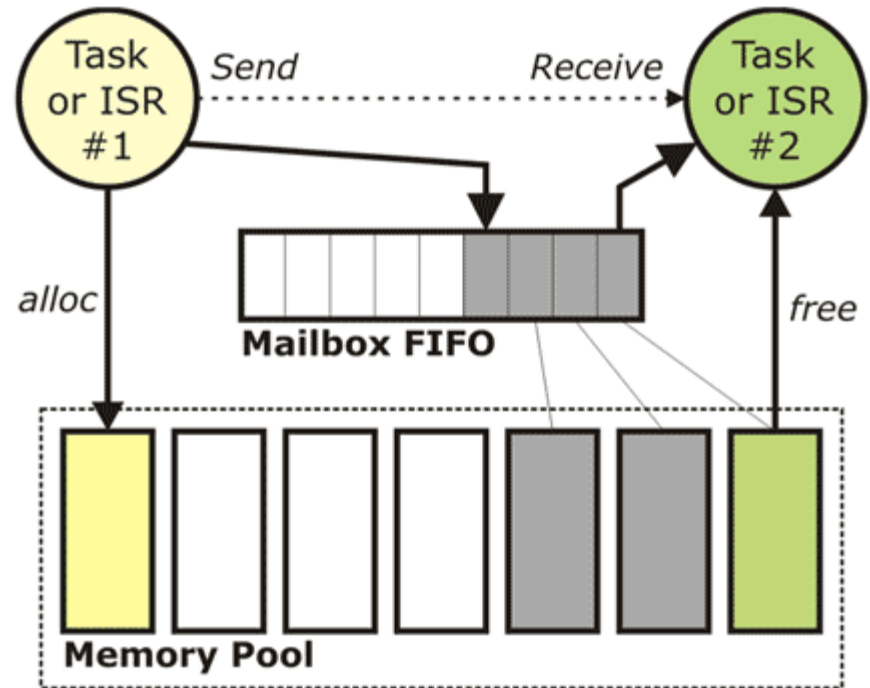


Mailboxes

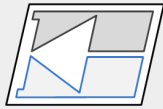
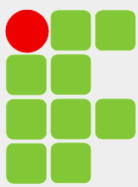


Mailbox é uma estrutura similar a fila de mensagens. É uma mistura de filas e semáforo, com alguns mecanismos de segurança que impedem escrita quando o sistema está cheio ou em uso.

Alguns RTOS permitem um certo número de mensagens por vez.



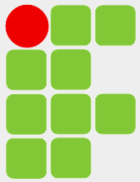
A mailbox consists of a memory block formatted into message buffers and a set of pointers to each buffer .



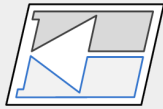
Estados de uma Tarefa

RUNNING	The currently running TASK
READY	TASKS ready to RUN
WAIT DELAY	TASK halted with a time DELAY
WAIT INT	TASKS scheduled to run periodically
WAIT OR	TASKS waiting an event flag to be set
WAIT AND	TASKS waiting for a group event flags to be set
WAIT SEM	TASKS waiting for a SEMAPHORE
WAIT MUT	TASKS waiting for a SEMAPHORE MUTEX
WAIT MBX	TASKS waiting for a SEMAPHORE MAILBOX MESSAGE
INACTIVE	A TASK not started or detected

At any given moment a single task may be running. The remaining tasks will be ready to run and will be scheduled by the kernel. Tasks may also be waiting pending an OS event. When this occurs they will return to the ready state and be scheduled by the kernel .

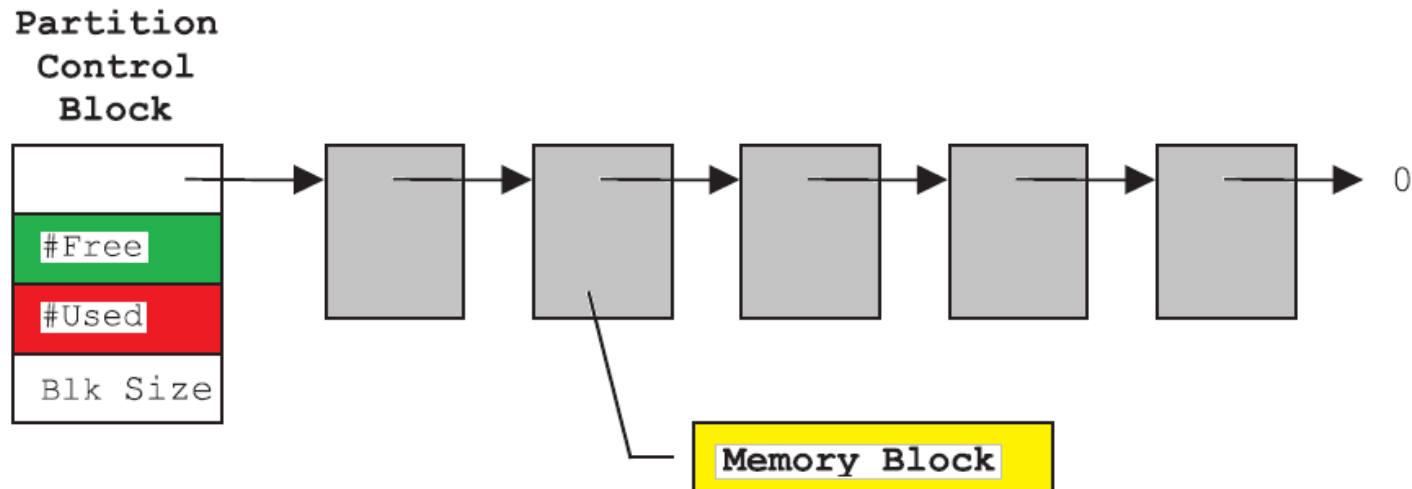


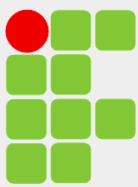
Gerenciamento de Memória



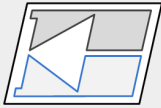
É possível utilizar as funções `malloc()` e `free()` para alocar e liberar memória (em C ANSI). Isso é perigoso em sistemas embarcados devido a fragmentação da memória impedindo o uso de áreas contíguas de memória, bem como o tempo de execução dessas funções não é determinístico.

Em contraposição, a maioria dos RTOS permite obter blocos de memória de tamanho fixo de uma partição em uma área contígua de memória. O *kernel* possui uma lista dos blocos livres de memória. A alocação e liberação desses blocos é feita em tempo constante e determinístico.





Exemplo Geral



Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}
```

Priority = 2

Period = 10

Execution time = 5

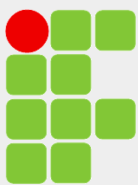
Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}
```

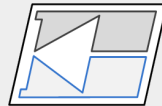
Priority = 1

Period = 50

Execution time = 10



Criação das Tarefas



Task Creation (main)

```
void main() {  
  
    char *mem1[100], *mem2[100];  
    int t1, t2;  
  
    /* create_task(@code, @param, @mem, priority */  
    t1 = create_task(*task1, NULL, mem1, 2);  
    t2 = create_task(*task2, NULL, mem2, 1);  
  
    /* OS starts executing */  
    os_start();  
} /* end */
```

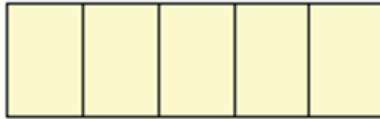
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

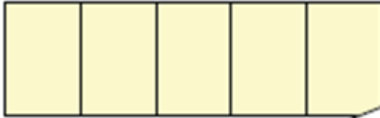
Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

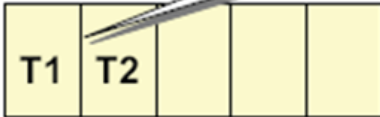
Blocked



Waiting



Ready

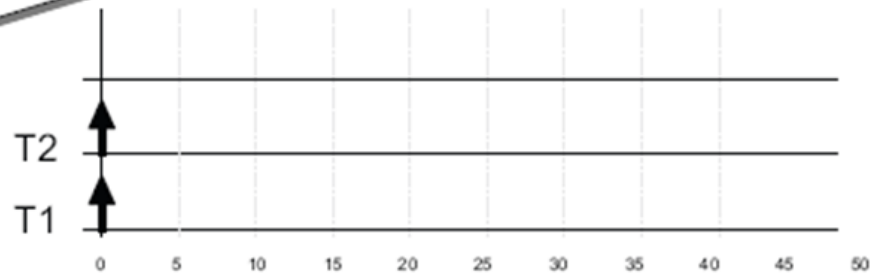


Running



PC =
Stack =
Mem =

Initially both tasks (T1 and T2) are ready to execute

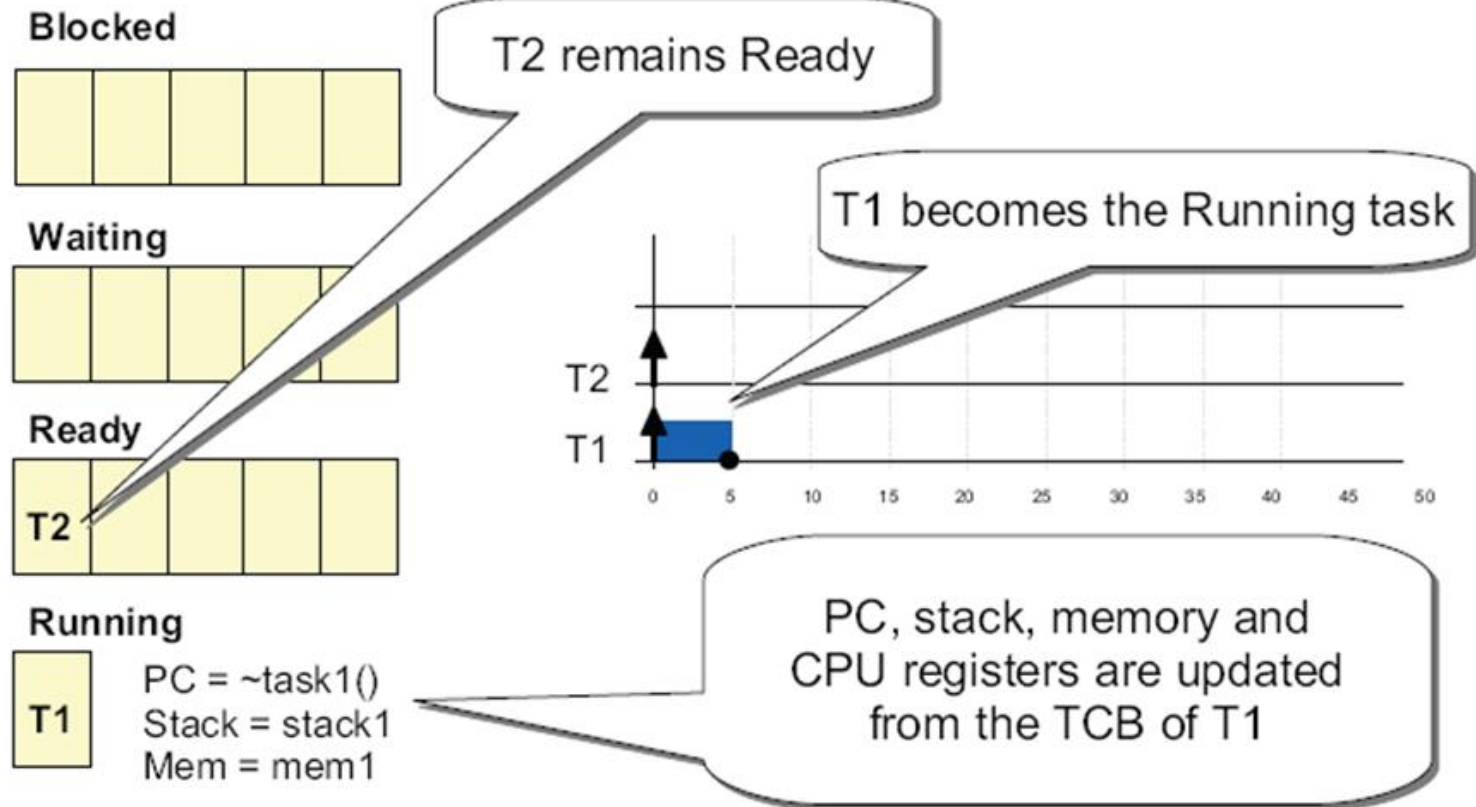


Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

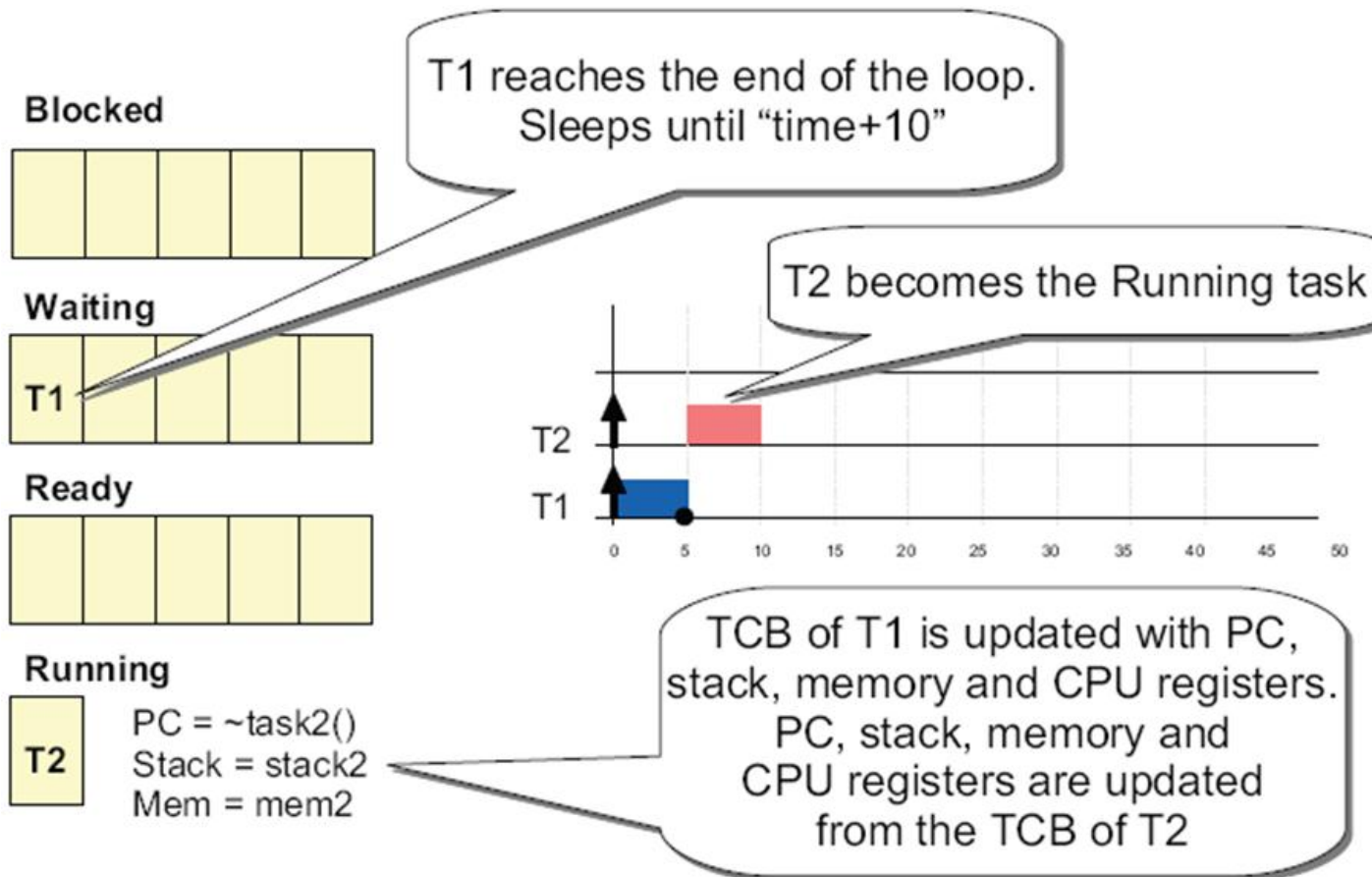


Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
    Priority = 2  
    Period = 10  
    Execution time = 5  
}
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
    Priority = 1  
    Period = 50  
    Execution time = 10  
}
```



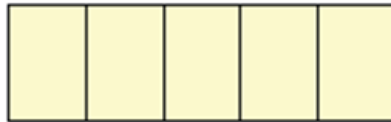
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

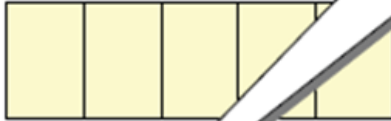
Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

Blocked



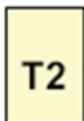
Waiting



Ready



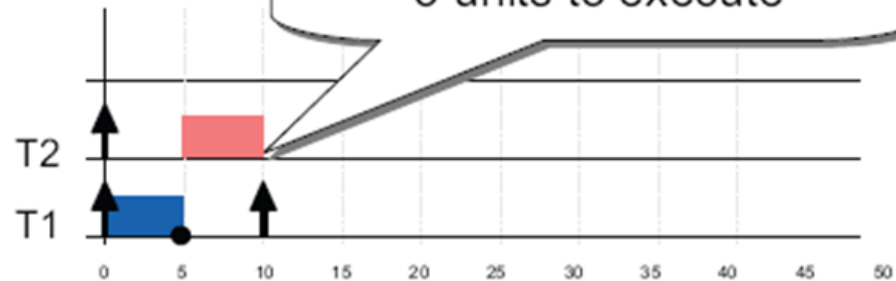
Running



PC = ~task2()
Stack = stack2
Mem = mem2

T1 becomes ready again

At this point T2 still has 5 units to execute

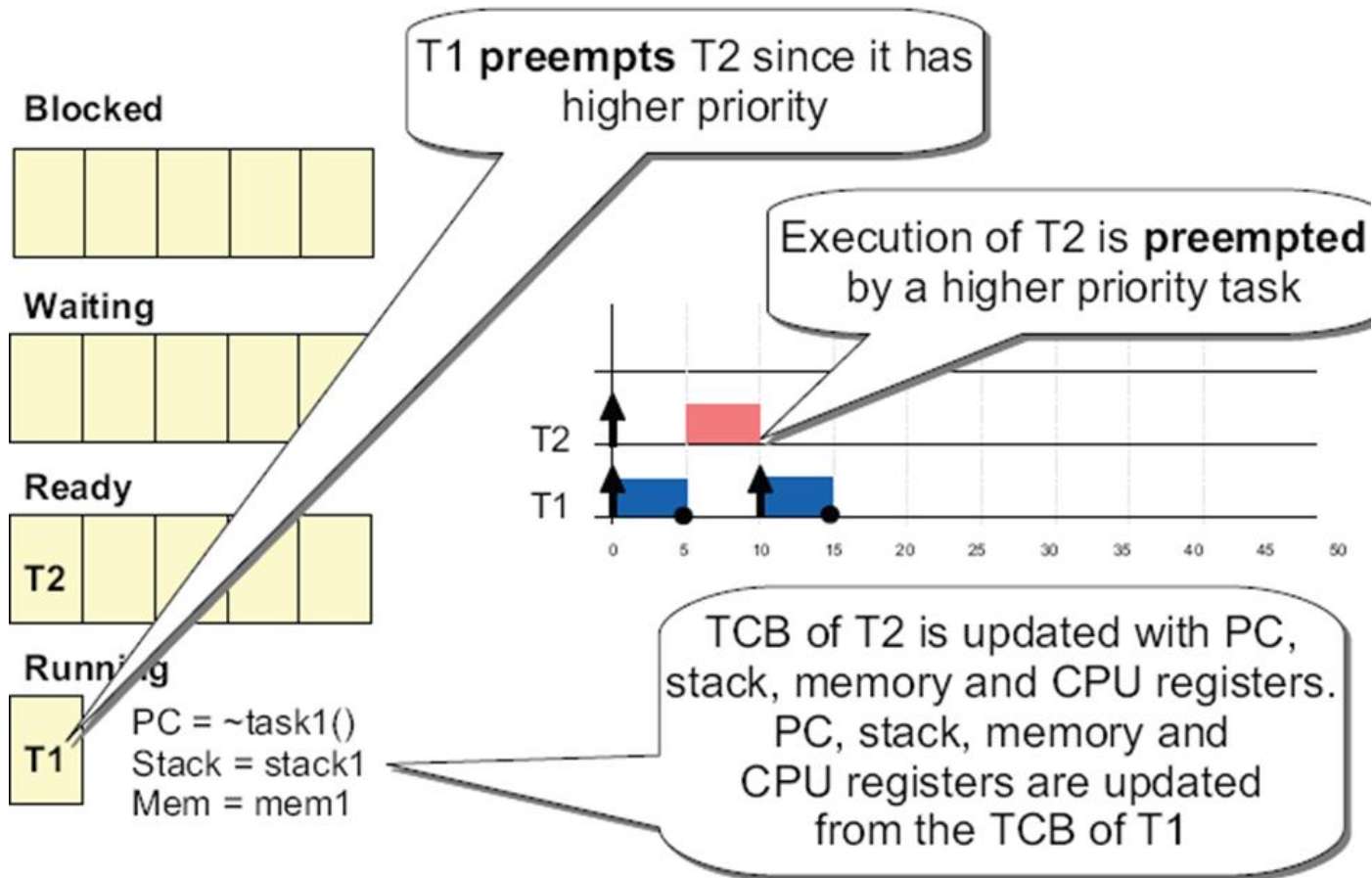


Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
    Priority = 2  
    Period = 10  
    Execution time = 5  
}
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
    Priority = 1  
    Period = 50  
    Execution time = 10  
}
```



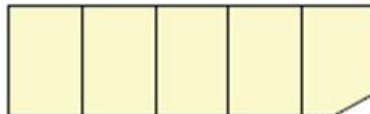
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
    Priority = 2  
    Period = 10  
    Execution time = 5  
}
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
    Priority = 1  
    Period = 50  
    Execution time = 10  
}
```

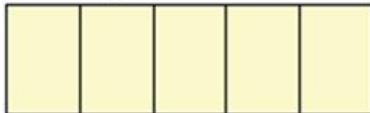
Blocked



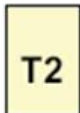
Waiting



Ready



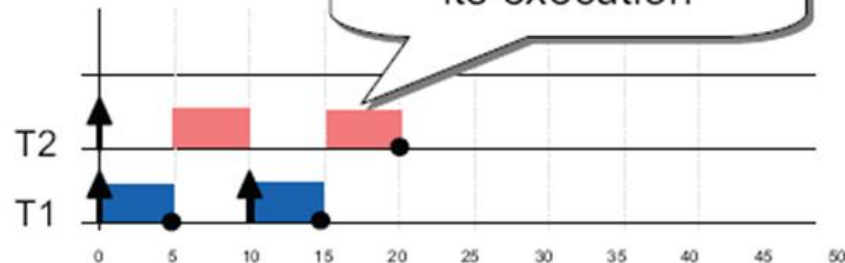
Running



PC = ~task2()
Stack = stack2
Mem = mem2

T1 reaches again the end of the loop and finishes its execution

T2 can now continue its execution



TCB of T2 is updated with PC, stack, memory and CPU registers. PC, stack, memory and CPU registers are updated from the TCB of T1

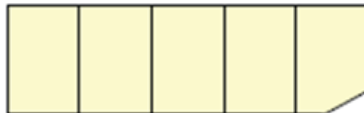
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

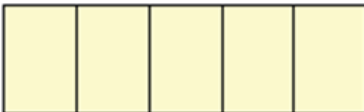
Blocked



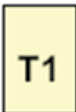
Waiting



Ready



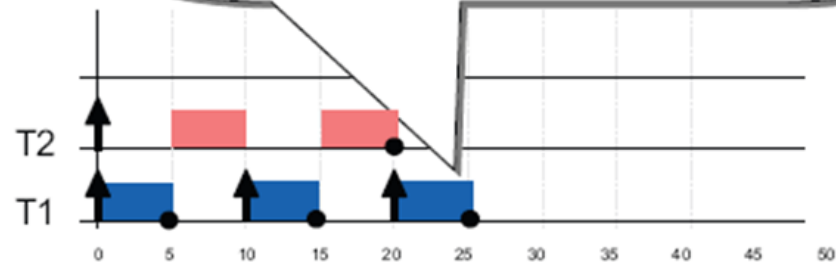
Running



PC = ~task1()
Stack = stack1
Mem = mem1

T2 reaches the end of its loop and finishes its execution

T1 executes again (after moving from Waiting to Ready)



TCB of T2 is updated with PC, stack, memory and CPU registers. PC, stack, memory and CPU registers are updated from the TCB of T1

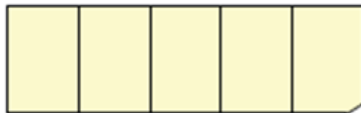
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
    Priority = 2  
    Period = 10  
    Execution time = 5  
}
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
    Priority = 1  
    Period = 50  
    Execution time = 10  
}
```

Blocked



Waiting



Ready



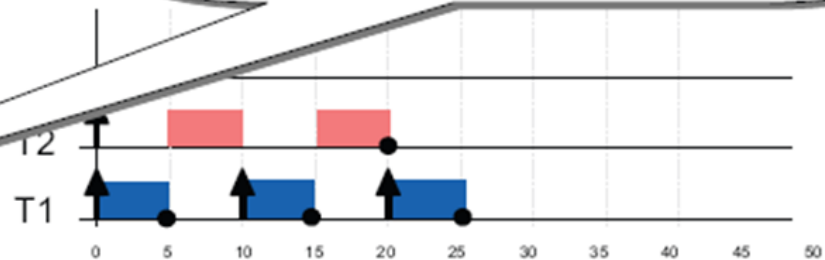
Running



PC = ??
Stack = ??
Mem = ??

T1 reaches the end of its loop and finishes its execution

No task is ready to be executed!!



And now what

?

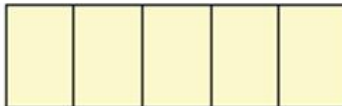
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

Blocked



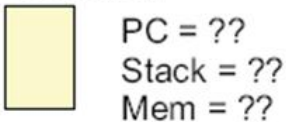
Waiting



Ready



Running



A special task **IDLE** is scheduled with the lowest priority in order to get the CPU when no other task is ready to execute

Task IDLE

```
void idle(void *param) {  
    while (1) { NULL }  
}
```

Priority = Lowest
Always Ready to execute

40 45 50

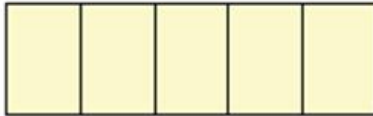
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

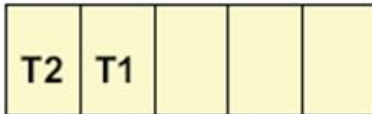
Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

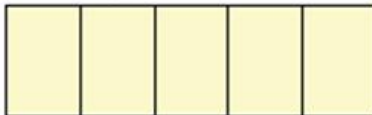
Blocked



Waiting



Ready

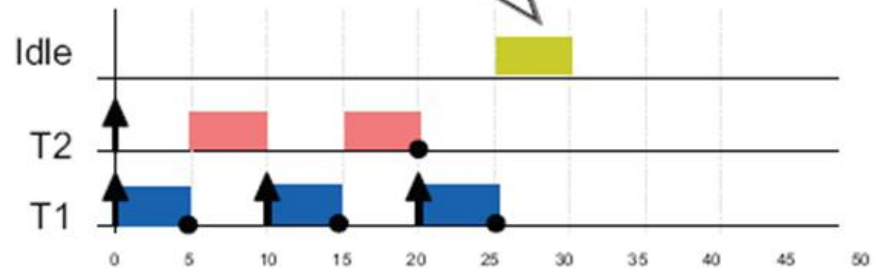


Running



PC = ~idle()
Stack = stack_idle
Mem = mem_idle

The Idle task executes until any other task is available in the Ready queue



Again PC, stack and memory are replaced like with any other task

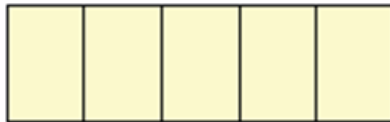
Task 1 (T1)

```
void task1(void *param) {  
    while (1) {  
        ...  
        echo("task 1");  
        sleep_until(time+10);  
    }  
}  
Priority = 2  
Period = 10  
Execution time = 5
```

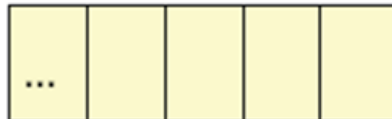
Task 2 (T2)

```
void task2(void *param) {  
    while (1) {  
        ...  
        echo("task 2");  
        sleep_until(time+50);  
    }  
}  
Priority = 1  
Period = 50  
Execution time = 10
```

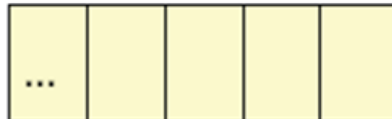
Blocked



Waiting



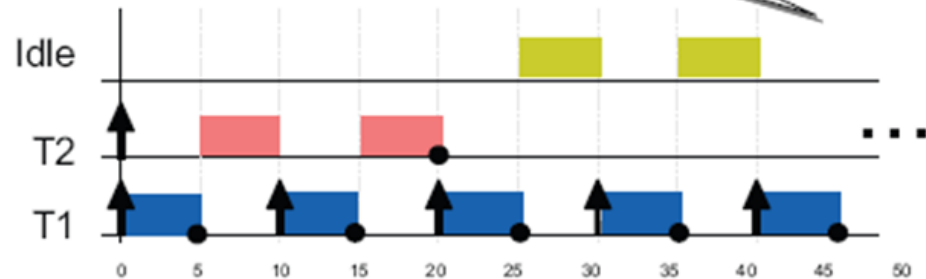
Ready

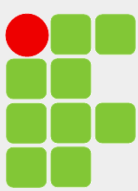


Running

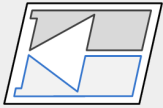


Execution continues...

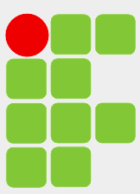




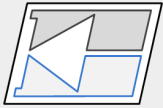
Comentários



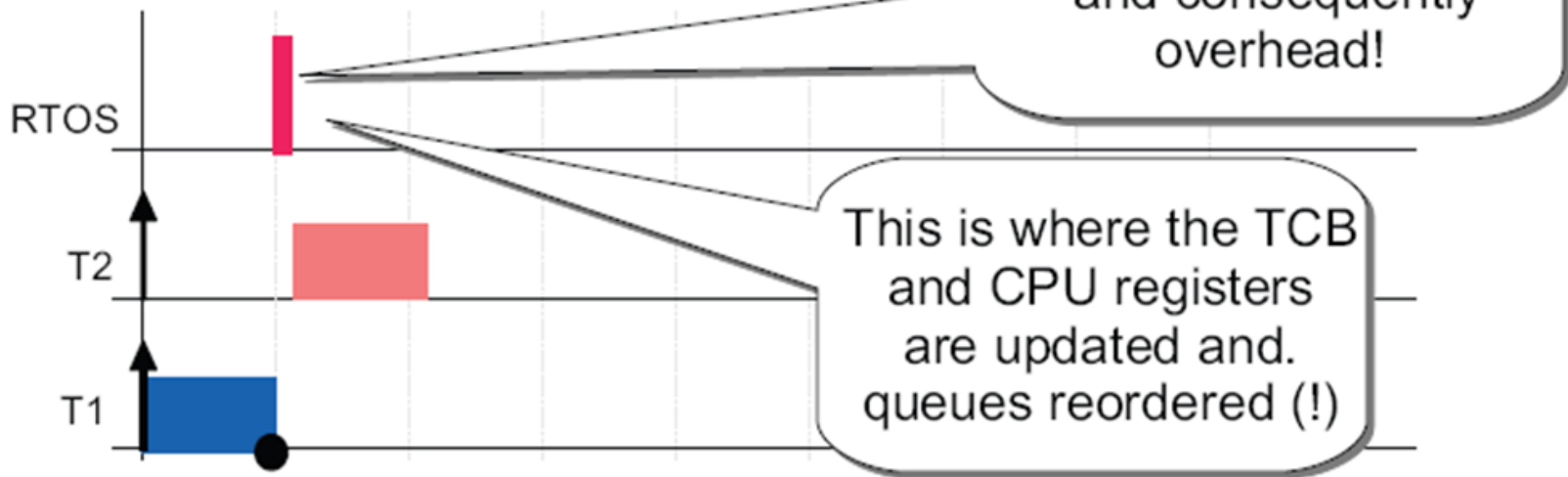
- Idle task is an infinite loop that gains CPU access whenever there is “nothing to do”.
- RTOS are **preemptive** since they “kick out” the current running task if there is a higher priority task ready to execute.
- Scheduling is based on selecting the appropriated task from (ordered) queues.
- Task context is saved on its own TCB and restored in every new execution.

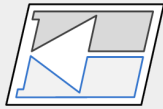
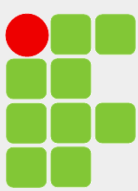


Comentários

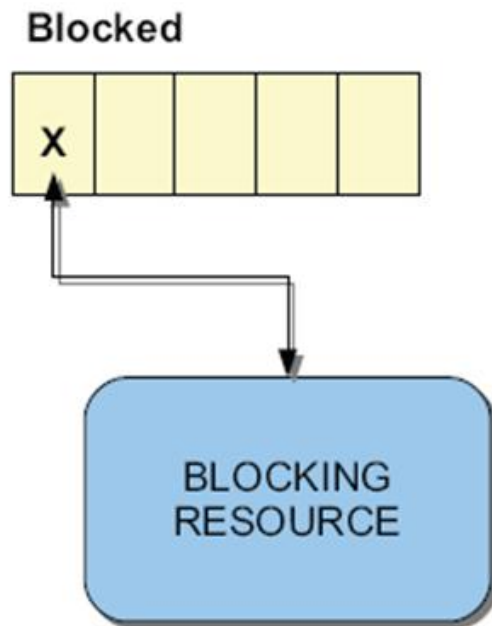


- Context switch might have effect:
 - Switching is not for free!:

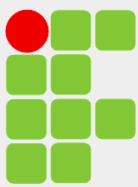




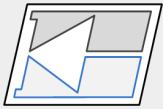
- What happens if tasks are blocked?



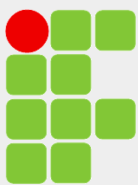
- Task goes to blocked queue
- A pointer is set to/from blocking resource
 - MUTEX, mailbox, etc...
- Task is moved to “Ready” when the resource is free



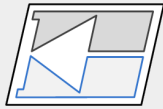
Selecionando um RTOS



- Popularidade — não é uma questão técnica, mas um indicativo.
- Reputação do vendedor.
- Código fonte — o código fonte é fornecido? Em alguns casos pode ser importante.
- Portabilidade.
- Escalabilidade — ajuste do gasto de memória (*footprint*).
- Preemptivo.
- Número de tarefas suportado.
- Tamanho de pilha ajustável por tarefa.
- Serviços necessários — semáforos, *timers*, *mailboxes*, filas, gerenciamento de memória...
- Desempenho — de 2% a 5% da CPU.
- Certificação.
- Ferramentas de desenvolvimento — IDEs.
- Componentes — bibliotecas de funções, TCP/IP, USB, CAN, sistema de arquivos...

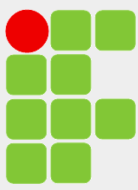


CONCLUSÃO

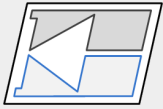


Desenvolver um programa embarcado de pequeno ou médio tamanho pode ser similar para programas com laço infinito /interrupções ou os que empregam um RTOS. Em muitos casos, quando não se emprega um RTOS, o programador deve escrever o equivalente a um gerenciador limitado para tratar os eventos. A vantagem de um RTOS é que o programador usa uma solução testada e confiável para realizar o mesmo objetivo. Adicionalmente, um RTOS possui características que fazem a estrutura de aplicações muito mais fácil de ser desenvolvida e mantida.

Em resumo, um RTOS é imprescindível para sistemas embarcados. Permite priorizar o trabalho que precisa ser feito em um determinado período de tempo por tarefas separadas, fornecendo serviços para o gerenciamento e interação entre as tarefas. A resposta de tarefas de alta prioridade é virtualmente não afetada quando tarefas de baixa prioridade são acrescentadas.



BIBLIOGRAFIA



GANSSELE, J. ***The Firmware Handbook: The Definitive Guide to Embedded Firmware Design and Applications***. 1a ed. Elsevier, 2004.

Yiu, Joseph. ***The Definitive Guide to ARM Cortex-M4 and Cortex-M4 Processors***. 3a ed. Elsevier, 2014.

TANENBAUM, Andrew S. ***Modern Operating Systems***. 3a ed. Pearson, 2008.

LIMA, Charles B. e Marco V.M. Villaça. ***AVR e Arduino: Técnicas de Projeto***. 2a ed. Edição dos Autores, 2012.

Real Time Operating Systems – RTOS – Ramon Serna Oliver, Kaiserslautern University of Technology.

The Insider's Guide to the NXP LPC2300/LCP2400 based Microcontroller - Hitex